

SOLIDWORKS API (.NET VBA C#) — Examples (45/50): Roll Back Model Example VB.NET Select Assembly Components by Size Examp

SOLIDWORKS API Documentation (converted from CHM)

Roll Back Model Example (VB.NET)

This example shows how to step through the FeatureManager design tree of a model by rolling back to each feature in reverse sequence. Running an example like this can provide insight into the design intent of the user.

```
'-----  
' Preconditions:  
' 1. Open a part or assembly document.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Observe the FeatureManager design tree  
'    while the macro executes.  
' 2. Examine the Immediate window to see the  
'    names of the features rolled back and forward.  
'  
' NOTE: The delay between steps is set to 1 second.  
'-----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System  
Imports System.Diagnostics  
Imports System.Windows.Forms.Application
```

```
Partial Class SolidWorksMacro
```

```
    Public Sub Main()
```

```
        ' Delay in seconds  
        Const DELAY As Single = 1.0#  
        Dim swModel As ModelDoc2  
        Dim swModelView As ModelView  
        Dim swPart As PartDoc  
        Dim swAssy As AssemblyDoc  
        Dim swFeatMgr As FeatureManager  
        Dim swFeat As Feature  
        Dim featFace As Object  
        Dim swFace As Face2  
        Dim rect As Object  
        Dim featName() As String  
        Dim timeNow As Single  
        Dim docType As Integer  
        Dim i As Integer  
        Dim j As Integer  
        Dim status As Boolean  
  
        swModel = swApp.ActiveDoc  
        swModelView = swModel.ActiveView  
        rect = Nothing  
        swFeatMgr = swModel.FeatureManager  
  
        swFeat = swModel.FirstFeature  
        docType = swModel.GetType  
        Select Case docType  
            Case swDocumentTypes_e.swDocPART  
                swPart = swModel  
            Case swDocumentTypes_e.swDocASSEMBLY  
                swAssy = swModel  
            Case Else  
                Debug.Print("Open a part or assembly document, then rerun this macro.")  
                Exit Sub  
        End Select  
  
        ReDim featName(0)  
  
        Do While Not swFeat Is Nothing  
            featName(UBound(featName)) = swFeat.Name  
            ReDim Preserve featName(UBound(featName) + 1)  
            swFeat = swFeat.GetNextFeature  
        Loop  
  
        ' Loop over-allocates array by 1, so remove last (empty) entry  
        ReDim Preserve featName(UBound(featName) - 1)  
  
        ' Playback each feature in the FeatureManager design tree  
        For i = 0 To UBound(featName)
```

```

Debug.Print(featName(i))
status = swFeatMgr.EditRollback(swMoveRollbackBarTo_e.swMoveRollbackBarToAfterFeature, featName(i))
' Do not assert because you may be trying to roll back or roll forward
' to a feature that cannot be rolled back or forward to; for example,
' the Lighting or Annotations folder
Debug.Assert status

' Remove any previous highlights
swModelView.GraphicsRedraw(rect)
' Highlight feature if it has any geometry
Select Case docType
    Case swDocumentTypes_e.swDocPART
        swPart = swModel
        swFeat = swPart.FeatureByName(featName(i))
    Case swDocumentTypes_e.swDocASSEMBLY
        swAssy = swModel
        swFeat = swAssy.FeatureByName(featName(i))
End Select

featFace = swFeat.GetFaces
If (isArray(featFace)) Then
    For j = 0 To (UBound(featFace) + 1)
        swFace = featFace(j)
        swFace.Highlight(True)
    Next j
End If

' Only pause if rollback is successful
If status Then
    timeNow = Timer
    While timeNow + DELAY > Timer
        ' Allow SOLIDWORKS Design to refresh screen
        DoEvents()
    End While
End If
Next i
' Remove highlight from last feature
swModelView.GraphicsRedraw(rect)

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Roll Model View Example (VBA)

This example shows how to roll a model view by a specified angle.

```
'-----  
' Preconditions: Open a part or assembly.  
'  
' Postconditions:  
' 1. Rolls the model view 90 degrees.  
' 2. Examine the graphics area.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelView As SldWorks.ModelView
```

Sub main()

```
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swModelView = swModel.ActiveView
```

```
    ' Roll the model view by 90 degrees  
    swModelView.RollBy (90#)
```

End Sub

Rotate and Copy 3D Sketch About Coordinates Example (C#)

This example shows how to rotate and copy 3D sketches.

```
//-----
// Preconditions:
// 1. Open or create a part document with two 3D sketches
//    named 3DSketch1 and 3DSketch2.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Copies and rotates 3DSketch2 around
//    the center point of 3DSketch1's arc.
// 2. Rotates 3DSketch1 around the center point of
//    its arc.
// 3. Examine the FeatureManager design tree and the
//    Immediate window.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;
using System.Windows.Forms;

namespace RotateOrCopy3DAboutXYZCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SelectData swSelData = default(SelectData);
            SketchManager swSketchMgr = default(SketchManager);
            Sketch swSketch = default(Sketch);
            SketchSegment swSketchSegment = default(SketchSegment);
            bool boolStatus = false;
            object[] varSketchSegments = null;
            int i = 0;

            // If SOLIDWORKS Design is not running, then exit macro
            if (swApp == null)
                return;

            // Document with two 3D sketches, named 3DSketch2 and
            // 3DSketch1, is open and active
            swModel = (ModelDoc2)swApp.ActiveDoc;
            if (swModel == null)
            {
                MessageBox.Show("Failed to open document.");
                return;
            }

            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swSelData = (SelectData)swSelMgr.CreateSelectData();
            swSketchMgr = (SketchManager)swModel.SketchManager;

            // Select 3DSketch2
            boolStatus = swModelDocExt.SelectByID2("3DSketch2", "SKETCH", 0, 0, 0, false, 0, null, 0);
            if (boolStatus == false)
            {
                MessageBox.Show("Failed to select 3DSketch2.");
                return;
            }

            // Open 3DSketch2 in edit mode
            swModel.EditSketch();
            swSketch = (Sketch)swSketchMgr.ActiveSketch;
            if (swSketch == null)
            {
                MessageBox.Show("Failed to get pointer to 3DSketch2.");
                return;
            }
        }
    }
}
```

```

// Select all sketch segments in 3DSketch2
varSketchSegments = (object[])swSketch.GetSketchSegments();
for (i = 0; i < varSketchSegments.Length; i++)
{
    swSketchSegment = (SketchSegment)varSketchSegments[i];
    boolStatus = swSketchSegment.Select4(true, swSelData);
    if (boolStatus == false)
        MessageBox.Show("Failed to select sketch segment instance." + i + ".");
}

// Copy and rotate 3DSketch2 about center
// point of 3DSketch1's arc
Debug.Print("Rotating and copying 3DSketch2 about the center point of 3DSketch1's arc? " + swSketchMgr.
RotateOrCopy3DAboutXYZ(true, 1, true, -0.09925811702374, 0.004131001848179, 0, 1.5707963267949, 0, 0));
swModel.ClearSelection2(true);

// Exit 3DSketch2
swSketchMgr.InsertSketch(true);

// Select 3DSketch1
boolStatus = swModelDocExt.SelectByID2("3DSketch1", "SKETCH", 0, 0, 0, false, 0, null, 0);
if (boolStatus == false)
{
    MessageBox.Show("Failed to select 3DSketch1.");
    return;
}

// Edit 3DSketch1
swModel.EditSketch();
swSketch = (Sketch)swModel.GetActiveSketch2();
if (swSketch == null)
{
    MessageBox.Show("Failed to get pointer to 3DSketch1.");
    return;
}

// Select all sketch segments in 3DSketch1
varSketchSegments = (object[])swSketch.GetSketchSegments();
for (i = 0; i < varSketchSegments.Length; i++)
{
    swSketchSegment = (SketchSegment)varSketchSegments[i];
    boolStatus = swSketchSegment.Select4(true, swSelData);
    if (boolStatus == false)
    {
        MessageBox.Show("Failed to select sketch segment instance." + i + ".");
        return;
    }
}

// Rotate 3DSketch1 about the
// center point of its arc
Debug.Print("Rotating 3DSketch1 about the center point of its arc? " + swSketchMgr.RotateOrCopy3DAboutX
YZ(false, 1, true, -0.09925811702374, 0.004131001848179, 0, 1.5707963267949, 0, 0));
swModel.ClearSelection2(true);

// Exit 3DSketch1
swSketchMgr.InsertSketch(true);
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Rotate and Copy 3D Sketch About Coordinates Example (VBA)

This example shows how to rotate and copy 3D sketches.

```
'-----  
' Preconditions:  
' 1. Open or create a part document with two 3D sketches  
'    named 3DSketch1 and 3DSketch2.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Copies and rotates 3DSketch2 around  
'    the center point of 3DSketch1's arc.  
' 2. Rotates 3DSketch1 around the center point of  
'    its arc.  
' 3. Examine the FeatureManager design tree and the  
'    Immediate window.  
'-----
```

Option Explicit

Sub main()

```
    Dim SwApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swModelDocExt As SldWorks.ModelDocExtension  
    Dim swSelData As SldWorks.SelectData  
    Dim swSketchMgr As SldWorks.SketchManager  
    Dim swSketch As SldWorks.Sketch  
    Dim boolStatus As Boolean  
    Dim varSketchSegments As Variant  
    Dim i As Integer  
  
    Set SwApp = Application.SldWorks  
  
    ' If SOLIDWORKS Design is not running, then exit macro  
    If SwApp Is Nothing Then Exit Sub  
  
    ' Document with two 3D sketches, named 3DSketch2 and  
    ' 3DSketch1, is open and active  
    Set swModel = SwApp.ActiveDoc  
    If swModel Is Nothing Then  
        MsgBox "Failed to open document."  
        Exit Sub  
    End If  
  
    Set swModelDocExt = swModel.Extension  
    Set swSelMgr = swModel.SelectionManager  
    Set swSelData = swSelMgr.CreateSelectData  
    Set swSketchMgr = swModel.SketchManager  
  
    ' Select 3DSketch2  
    boolStatus = swModelDocExt.SelectByID2("3DSketch2", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)  
    If boolStatus = False Then  
        MsgBox "Failed to select 3DSketch2."  
        Exit Sub  
    End If  
  
    ' Open 3DSketch2 in edit mode  
    swModel.EditSketch  
    Set swSketch = swSketchMgr.ActiveSketch  
    If swSketch Is Nothing Then  
        MsgBox "Failed to get pointer to 3DSketch2."  
        Exit Sub  
    End If  
  
    ' Select all sketch segments in 3DSketch2  
    varSketchSegments = swSketch.GetSketchSegments()  
    For i = 0 To UBound(varSketchSegments)  
        boolStatus = varSketchSegments(i).Select4(True, swSelData)
```

```

        If boolStatus = False Then MsgBox "Failed to select sketch segment instance." & i & "."
    Next i

    ' Copy and rotate 3DSketch2 about center
    ' point of 3DSketch1's arc
    Debug.Print "Rotating and copying 3DSketch2 about the center point of 3DSketch1's arc? " &
swSketchMgr.RotateOrCopy3DAboutXYZ(True, 1, True, -0.09925811702374, 0.004131001848179, 0, 1.5707963267949, 0, 0)
    swModel.ClearSelection2 True

    ' Exit 3DSketch2
    swSketchMgr.InsertSketch True

    ' Select 3DSketch1
    boolStatus = swModelDocExt.SelectByID2("3DSketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
    If boolStatus = False Then
        MsgBox "Failed to select 3DSketch1."
        Exit Sub
    End If

    ' Edit 3DSketch1
    swModel.EditSketch
    Set swSketch = swModel.GetActiveSketch2
    If swSketch Is Nothing Then
        MsgBox "Failed to get pointer to 3DSketch1."
        Exit Sub
    End If

    ' Select all sketch segments in 3DSketch1
    varSketchSegments = swSketch.GetSketchSegments()
    For i = 0 To UBound(varSketchSegments)
        boolStatus = varSketchSegments(i).Select4(True, swSelData)
        If boolStatus = False Then
            MsgBox "Failed to select sketch segment instance." & i & "."
            Exit Sub
        End If
    Next i

    ' Rotate 3DSketch1 about the
    ' center point of its arc
    Debug.Print "Rotating 3DSketch1 about the center point of its arc? " &
swSketchMgr.RotateOrCopy3DAboutXYZ(False, 1, True, -0.09925811702374, 0.004131001848179, 0, 1.5707963267949, 0, 0)
    swModel.ClearSelection2 True

    ' Exit 3DSketch1
    swSketchMgr.InsertSketch True

End Sub

```


Rotate and Copy 3D Sketch About Coordinates Example (VB.NET)

This example shows how to rotate and copy 3D sketches.

```
'-----
' Preconditions:
' 1. Open or create a part document with two 3D sketches
'    named 3DSketch1 and 3DSketch2.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Copies and rotates 3DSketch2 around
'    the center point of 3DSketch1's arc.
' 2. Rotates 3DSketch1 around the center point of
'    its arc.
' 3. Examine the FeatureManager design tree and the
'    Immediate window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swSelMgr As SelectionMgr
        Dim swModelDocExt As ModelDocExtension
        Dim swSelData As SelectData
        Dim swSketchMgr As SketchManager
        Dim swSketch As Sketch
        Dim boolStatus As Boolean
        Dim varSketchSegments() As Object
        Dim i As Integer

        ' If SOLIDWORKS Design is not running, then exit macro
        If SwApp Is Nothing Then Exit Sub

        ' Document with two 3D sketches, named 3DSketch2 and
        ' 3DSketch1, is open and active
        swModel = SwApp.ActiveDoc
        If swModel Is Nothing Then
            MsgBox("Failed to open document.")
            Exit Sub
        End If

        swModelDocExt = swModel.Extension
        swSelMgr = swModel.SelectionManager
        swSelData = swSelMgr.CreateSelectData
        swSketchMgr = swModel.SketchManager

        ' Select 3DSketch2
        boolStatus = swModelDocExt.SelectByID2("3DSketch2", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
        If boolStatus = False Then
            MsgBox("Failed to select 3DSketch2.")
            Exit Sub
        End If

        ' Open 3DSketch2 in edit mode
        swModel.EditSketch()
        swSketch = swSketchMgr.ActiveSketch
        If swSketch Is Nothing Then
            MsgBox("Failed to get pointer to 3DSketch2.")
            Exit Sub
        End If

        ' Select all sketch segments in 3DSketch2
        varSketchSegments = swSketch.GetSketchSegments()
        For i = 0 To UBound(varSketchSegments)
            boolStatus = varSketchSegments(i).Select4(True, swSelData)
            If boolStatus = False Then MsgBox("Failed to select sketch segment instance." & i & ".")
        Next i

        ' Copy and rotate 3DSketch2 about center
        ' point of 3DSketch1's arc
```

```

        Debug.Print("Rotating and copying 3DSketch2 about the center point of 3DSketch1's arc? " & swSketchMgr.RotateOrCopy3DAboutXYZ(True, 1, True, -0.09925811702374, 0.004131001848179, 0, 1.5707963267949, 0, 0))
        swModel.ClearSelection2(True)

        ' Exit 3DSketch2
        swSketchMgr.InsertSketch(True)

        ' Select 3DSketch1
        boolStatus = swModelDocExt.SelectByID2("3DSketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
        If boolStatus = False Then
            MsgBox("Failed to select 3DSketch1.")
            Exit Sub
        End If

        ' Edit 3DSketch1
        swModel.EditSketch()
        swSketch = swModel.GetActiveSketch2
        If swSketch Is Nothing Then
            MsgBox("Failed to get pointer to 3DSketch1.")
            Exit Sub
        End If

        ' Select all sketch segments in 3DSketch1
        varSketchSegments = swSketch.GetSketchSegments()
        For i = 0 To UBound(varSketchSegments)
            boolStatus = varSketchSegments(i).Select4(True, swSelData)
            If boolStatus = False Then
                MsgBox("Failed to select sketch segment instance." & i & ".")
                Exit Sub
            End If
        Next i

        ' Rotate 3DSketch1 about the
        ' center point of its arc
        Debug.Print("Rotating 3DSketch1 about the center point of its arc? " & swSketchMgr.RotateOrCopy3DAboutXYZ(False, 1, True, -0.09925811702374, 0.004131001848179, 0, 1.5707963267949, 0, 0))
        swModel.ClearSelection2(True)

        ' Exit 3DSketch1
        swSketchMgr.InsertSketch(True)

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Rotate and Copy 3D Sketch About Vector Example (C#)

This example shows how to rotate and copy 3D sketches about a vector.

```
//-----  
// Preconditions:  
// 1. Open a part that contains two 3D sketches,  
//    3DSketch1 and 3DSketch2.  
// 2. Open the Immediate window.  
//  
// Postconditions:  
// 1. Copies and rotates the 3DSketch2 sketch around  
//    the center point of the 3DSketch1 sketch's arc.  
// 2. Rotates the 3DSketch1 sketch around the center  
//    point of its arc.  
// 3. Examine the Immediate window.  
//-----  
  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System;  
using System.Diagnostics;  
namespace RotateorCopy3DAboutVectorXYZ_CSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
        public void Main()  
        {  
            ModelDoc2 swModel = default(ModelDoc2);  
            SelectionMgr swSelMgr = default(SelectionMgr);  
            ModelDocExtension swModelDocExt = default(ModelDocExtension);  
            SelectData swSelData = default(SelectData);  
            SketchManager swSketchMgr = default(SketchManager);  
            Sketch swSketch = default(Sketch);  
            bool boolStatus = false;  
            object[] varSketchSegments = null;  
            int i = 0;  
            if (swApp == null) return;  
            // Document with two 3D sketches, named 3DSketch2 and  
            // 3DSketch1, is open and active  
            swModel = (ModelDoc2)swApp.ActiveDoc;  
            if (swModel == null)  
            {  
                System.Windows.Forms.MessageBox.Show("Failed to open document.");  
                return;  
            }  
            swModelDocExt = swModel.Extension;  
            swSelMgr = (SelectionMgr)swModel.SelectionManager;  
            swSelData = (SelectData)swSelMgr.CreateSelectData();
```

```

swSketchMgr = swModel.SketchManager;
// Select 3DSketch2 sketch
boolStatus = swModelDocExt.SelectByID2("3DSketch2", "SKETCH", 0, 0, 0, false, 0, null,
0);

if (boolStatus == false)
{
    System.Windows.Forms.MessageBox.Show("Failed to select sketch 3DSketch2.");
    return;
}
// Open 3DSketch2 sketch in edit mode
swModel.EditSketch();
swSketch = swSketchMgr.ActiveSketch;
if (swSketch == null)
{
    System.Windows.Forms.MessageBox.Show("Failed to get pointer to 3DSketch2
sketch.");
    return;
}
// Select all sketch segments in 3DSketch2 sketch
varSketchSegments = (object[])swSketch.GetSketchSegments();
for (i = 0; i <= varSketchSegments.GetUpperBound(0); i++)
{
    boolStatus = ((SketchSegment)varSketchSegments[i]).Select4(true, swSelData);
    if (boolStatus == false) System.Windows.Forms.MessageBox.Show("Failed to select
sketch segment instance." + i + ".");
}
// Copy and rotate 3DSketch2 sketch about center
// point of 3DSketch1 sketch's arc
Debug.Print("Rotating and copying 3DSketch2 sketch about the center point of
3DSketch1's arc? " + swSketchMgr.RotateOrCopy3DAboutVector(true, 1, true, -0.09925811702374,
0.004131001848179, 0, 1, 0, 0, 1.5707963267949
));
swModel.ClearSelection2(true);
// Exit 3DSketch2 sketch
swSketchMgr.InsertSketch(true);
// Select 3DSketch1 sketch
boolStatus = swModelDocExt.SelectByID2("3DSketch1", "SKETCH", 0, 0, 0, false, 0, null,
0);

if (boolStatus == false)
{
    System.Windows.Forms.MessageBox.Show("Failed to select sketch 3DSketch1.");
    return;
}
// Edit 3DSketch1 sketch
swModel.EditSketch();
swSketch = (Sketch)swModel.GetActiveSketch2();
if (swSketch == null)
{

```

```

3DSketch1.");
        System.Windows.Forms.MessageBox.Show("Failed to get pointer to sketch
        return;
    }
    // Select all sketch segments in 3DSketch1 sketch
    varSketchSegments = (object[])swSketch.GetSketchSegments();
    for (i = 0; i <= varSketchSegments.GetUpperBound(0); i++)
    {
        boolStatus = ((SketchSegment)varSketchSegments[i]).Select4(true, swSelData);
        if (boolStatus == false)
        {
            System.Windows.Forms.MessageBox.Show("Failed to select sketch segment
instance." + i + ".");
            return;
        }
    }
    // Rotate 3DSketch1 sketch about the
    // center point of its arc
    Debug.Print("Rotating 3DSketch1 sketch about the center point of its arc? " +
swSketchMgr.RotateOrCopy3DAboutVector(false, 1, true, -0.09925811702374, 0.004131001848179, 0, 1,
0, 0, 1.5707963267949
    ));
    swModel.ClearSelection2(true);
    // Exit 3DSketch1 sketch
    swSketchMgr.InsertSketch(true);
}
public SldWorks swApp;
}
}

```

Rotate and Copy 3D Sketch About Vector Example (VBA)

This example shows how to rotate and copy 3D sketches about a vector.

```
' -----  
' Preconditions:  
' 1. Open a part that contains two 3D sketches,  
'    3DSketch1 and 3DSketch2.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Copies and rotates the 3DSketch2 sketch around  
'    the center point of the 3DSketch1 sketch's arc.  
' 2. Rotates the 3DSketch1 sketch around the center  
'    point of its arc.  
' 3. Examine the Immediate window.  
' -----
```

Option Explicit

Sub main()

```
Dim SwApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swSelData As SldWorks.SelectData  
Dim swSketchMgr As SldWorks.SketchManager  
Dim swSketch As SldWorks.Sketch  
Dim boolStatus As Boolean  
Dim varSketchSegments As Variant  
Dim i As Integer  
  
Set SwApp = Application.SldWorks  
' If SOLIDWORKS Design is not running, then exit macro  
If SwApp Is Nothing Then Exit Sub  
  
' Document with two 3D sketches, named 3DSketch2 and  
' 3DSketch1, is open and active  
Set swModel = SwApp.ActiveDoc  
If swModel Is Nothing Then  
    MsgBox "Failed to open document."  
    Exit Sub  
End If  
  
Set swModelDocExt = swModel.Extension  
Set swSelMgr = swModel.SelectionManager  
Set swSelData = swSelMgr.CreateSelectData  
Set swSketchMgr = swModel.SketchManager
```

```

' Select 3DSketch2 sketch
boolStatus = swModelDocExt.SelectByID2("3DSketch2", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
If boolStatus = False Then
    MsgBox "Failed to select 3DSketch2 sketch."
    Exit Sub
End If

' Open 3DSketch2 sketch in edit mode
swModel.EditSketch
Set swSketch = swSketchMgr.ActiveSketch
If swSketch Is Nothing Then
    MsgBox "Failed to get pointer to 3DSketch2 sketch."
    Exit Sub
End If

' Select all sketch segments in 3DSketch2 sketch
varSketchSegments = swSketch.GetSketchSegments()
For i = 0 To UBound(varSketchSegments)
    boolStatus = varSketchSegments(i).select4(True, swSelData)
    If boolStatus = False Then MsgBox "Failed to select sketch segment instance." & i & "."
Next i

' Copy and rotate 3DSketch2 sketch about center
' point of 3DSketch1 sketch's arc
Debug.Print "Rotating and copying 3DSketch2 sketch about the center point of 3DSketch1's arc?"
" & swSketchMgr.RotateOrCopy3DAboutVector(True, 1, True, -0.09925811702374, 0.004131001848179, 0,
1, 0, 0, 1.5707963267949)
swModel.ClearSelection2 True
' Exit 3DSketch2 sketch
swSketchMgr.InsertSketch True

' Select 3DSketch1 sketch
boolStatus = swModelDocExt.SelectByID2("3DSketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
If boolStatus = False Then
    MsgBox "Failed to select 3DSketch1 sketch."
    Exit Sub
End If

' Edit 3DSketch1 sketch
swModel.EditSketch
Set swSketch = swModel.GetActiveSketch2
If swSketch Is Nothing Then
    MsgBox "Failed to get pointer to 3DSketch1 sketch."
    Exit Sub
End If

```

```

' Select all sketch segments in 3DSketch1 sketch
varSketchSegments = swSketch.GetSketchSegments()
For i = 0 To UBound(varSketchSegments)
    boolStatus = varSketchSegments(i).select4(True, swSelData)
    If boolStatus = False Then
        MsgBox "Failed to select sketch segment instance." & i & "."
        Exit Sub
    End If
Next i

' Rotate 3DSketch1 sketch about the
' center point of its own arc
Debug.Print "Rotating 3DSketch1 sketch about the center point of its arc? " &
swSketchMgr.RotateOrCopy3DAboutVector(False, 1, True, -0.09925811702374, 0.004131001848179, 0, 1,
0, 0, 1.5707963267949)
swModel.ClearSelection2 True

' Exit 3DSketch1 sketch
swSketchMgr.InsertSketch True

End Sub

```


Rotate and Copy 3D Sketch About Vector Example (VB.NET)

This example shows how to rotate and copy 3D sketches about a vector.

```
'-----  
' Preconditions:  
' 1. Open a part that contains two 3D sketches,  
'    3DSketch1 and 3DSketch2.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Copies and rotates the 3DSketch2 sketch around  
'    the center point of the 3DSketch1 sketch's arc.  
' 2. Rotates the 3DSketch1 sketch around the center  
'    point of its arc.  
' 3. Examine the Immediate window.  
'-----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System
```

```
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Public Sub main()
```

```
        Dim swModel As ModelDoc2
```

```
        Dim swSelMgr As SelectionMgr
```

```
        Dim swModelDocExt As ModelDocExtension
```

```
        Dim swSelData As SelectData
```

```
        Dim swSketchMgr As SketchManager
```

```
        Dim swSketch As Sketch
```

```
        Dim boolStatus As Boolean
```

```
        Dim varSketchSegments As Object
```

```
        Dim i As Integer
```

```
        ' If SOLIDWORKS isn't running,
```

```
        ' then exit
```

```
        If swApp Is Nothing Then Exit Sub
```

```
        ' Document with two 3D sketches, named 3DSketch2 and
```

```
        ' 3DSketch1, is open and active
```

```
        swModel = swApp.ActiveDoc
```

```
        If swModel Is Nothing Then
```

```
            MsgBox("Failed to open document.")
```

```
            Exit Sub
```

```
        End If
```

```

swModelDocExt = swModel.Extension
swSelMgr = swModel.SelectionManager
swSelData = swSelMgr.CreateSelectData
swSketchMgr = swModel.SketchManager

' Select 3DSketch2 sketch
boolStatus = swModelDocExt.SelectByID2("3DSketch2", "SKETCH", 0, 0, 0, False, 0, Nothing,
0)

If boolStatus = False Then
    MsgBox("Failed to select sketch 3DSketch2.")
    Exit Sub
End If

' Open 3DSketch2 sketch in edit mode
swModel.EditSketch()
swSketch = swSketchMgr.ActiveSketch
If swSketch Is Nothing Then
    MsgBox("Failed to get pointer to 3DSketch2 sketch.")
    Exit Sub
End If

' Select all sketch segments in 3DSketch2 sketch
varSketchSegments = swSketch.GetSketchSegments()
For i = 0 To UBound(varSketchSegments)
    boolStatus = varSketchSegments(i).select4(True, swSelData)
    If boolStatus = False Then MsgBox("Failed to select sketch segment instance." & i &
".")
Next i

' Copy and rotate 3DSketch2 sketch about center
' point of 3DSketch1 sketch's arc
Debug.Print("Rotating and copying 3DSketch2 sketch about the center point of 3DSketch1's
arc? " & swSketchMgr.RotateOrCopy3DAboutVector(True, 1, True, -0.09925811702374,
0.004131001848179, 0, 1, 0, 0, 1.5707963267949))
swModel.ClearSelection2(True)

' Exit 3DSketch2 sketch
swSketchMgr.InsertSketch(True)

' Select 3DSketch1 sketch
boolStatus = swModelDocExt.SelectByID2("3DSketch1", "SKETCH", 0, 0, 0, False, 0, Nothing,
0)

If boolStatus = False Then
    MsgBox("Failed to select sketch 3DSketch1.")
    Exit Sub
End If

' Edit 3DSketch1 sketch
swModel.EditSketch()

```

```

swSketch = swModel.GetActiveSketch2
If swSketch Is Nothing Then
    MsgBox("Failed to get pointer to sketch 3DSketch1.")
    Exit Sub
End If

' Select all sketch segments in 3DSketch1 sketch
varSketchSegments = swSketch.GetSketchSegments()
For i = 0 To UBound(varSketchSegments)
    boolStatus = varSketchSegments(i).select4(True, swSelData)
    If boolStatus = False Then
        MsgBox("Failed to select sketch segment instance." & i & ".")
        Exit Sub
    End If
Next i

' Rotate 3DSketch1 sketch about the
' center point of its arc
Debug.Print("Rotating 3DSketch1 sketch about the center point of its arc? " &
swSketchMgr.RotateOrCopy3DAboutVector(True, 1, True, -0.09925811702374, 0.004131001848179, 0, 1,
0, 0, 1.5707963267949))
swModel.ClearSelection2(True)
' Exit 3DSketch1 sketch
swSketchMgr.InsertSketch(True)

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Rotate Assembly Component on Axis Using IDragOperator::DragAsUI Example (C#)

This example shows how to rotate an assembly component about an assembly axis using IDragOperator::DragAsUI.

```
//-----
// Preconditions:
// 1. Assembly document is open.
// 2. Component in the assembly document is selected.
//
// Postconditions: Transform matrix is set.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;
namespace DragOperatorAddComponent_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        const double PI = 3.14159;
        const double RadPerDeg = PI / 180.0;

        // DragOperator::TransformType
        // Translation      0
        // Transform is translation-only.
        //
        // Axial rotation   1
        // Transform is rotation-only.
        //
        // General          2
        // Transform can be translation, rotation, or both.
        //
        // DragOperator::DragMode
        // Minimum Move     0
        // Move smallest number of geometries.
        //
        // Maximum Move     1
        // Move geometries rigidly if possible.
        //
        // Relaxation        2
        // Solve geometries using relaxation.

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            AssemblyDoc swAssy = default(AssemblyDoc);
            DragOperator swDragOp = default(DragOperator);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Component2[] swComp = new Component2[2];
            object vComp = null;
            MathTransform swXform = default(MathTransform);
            MathUtility swMathUtil = default(MathUtility);
            MathPoint swOriginPt = default(MathPoint);
            MathVector swX_Axis = default(MathVector);
            double[] nPts = new double[3];
            object vData = null;
            int i = 0;
            bool bRet = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swAssy = (AssemblyDoc)swModel;
            swDragOp = (DragOperator)swAssy.GetDragOperator();
        }
    }
}
```

```

swSelMgr = (SelectionMgr)swModel.SelectionManager;
swComp[0] = (Component2)swSelMgr.GetSelectedObjectsComponent(1);
swComp[1] = (Component2)swSelMgr.GetSelectedObjectsComponent(2);
swMathUtil = (MathUtility)swApp.GetMathUtility();

vComp = swComp;

nPts[0] = 0.0;
nPts[1] = 0.0;
nPts[2] = 0.0;

vData = nPts;

swOriginPt = (MathPoint)swMathUtil.CreatePoint(vData);

nPts[0] = 1.0;
nPts[1] = 0.0;
nPts[2] = 0.0;
vData = nPts;

swX_Axis = (MathVector)swMathUtil.CreateVector(vData);

// This is an incremental rotation,
// so angle is always the same.
swXform = (MathTransform)swMathUtil.CreateTransformRotateAxis(swOriginPt, swX_Axis,
1.0 * RadPerDeg);

bRet = swDragOp.AddComponent(swComp[0], false);
Debug.Assert(bRet);

swDragOp.DynamicClearanceEnabled = false;

// Axial rotation
swDragOp.TransformType = 1;

// Solve by relaxation
swDragOp.DragMode = 2;

bRet = swDragOp.BeginDrag();
Debug.Assert(bRet);

for (i = 0; i <= 500; i++)
{
    bRet = swDragOp.DragAsUI(swXform);
}

bRet = swDragOp.EndDrag();
Debug.Assert(bRet);
}

public SldWorks swApp;
}
}

```

Rotate Assembly Component on Axis Using IDragOperator::DragAsUI Example (VBA)

This example shows how to rotate an assembly component about an assembly axis using IDragOperator::DragAsUI.

```
'-----  
' Preconditions:  
' 1. Assembly document is open.  
' 2. Component in the assembly document is selected.  
'  
' Postconditions: Transform matrix is set.  
'-----  
  
Option Explicit  
Const PI As Double = 3.14159  
Const RadPerDeg As Double = PI / 180#  
  
' DragOperator::TransformType  
' Translation 0  
' Transform is translation-only.  
'  
' Axial rotation 1  
' Transform is rotation-only.  
'  
' General 2  
' Transform can be translation, rotation, or both.  
'  
' DragOperator::DragMode  
' Minimum Move 0  
' Move smallest number of geometries.  
'  
' Maximum Move 1  
' Move geometries rigidly if possible.  
'  
' Relaxation 2  
' Solve geometries using relaxation.
```

```
Sub main()  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swAssy As SldWorks.AssemblyDoc  
Dim swDragOp As SldWorks.DragOperator  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swComp(1) As SldWorks.Component2  
Dim vComp As Variant  
Dim swXform As SldWorks.MathTransform  
Dim swMathUtil As SldWorks.MathUtility  
Dim swOriginPt As SldWorks.MathPoint  
Dim swX_Axis As SldWorks.MathVector  
Dim nPts(2) As Double  
Dim vData As Variant  
Dim i As Long  
Dim bRet As Boolean  
  
Set swApp = CreateObject("SldWorks.Application")  
Set swModel = swApp.ActiveDoc  
Set swAssy = swModel  
Set swDragOp = swAssy.GetDragOperator  
Set swSelMgr = swModel.SelectionManager  
Set swComp(0) = swSelMgr.GetSelectedObjectsComponent(1)  
Set swComp(1) = swSelMgr.GetSelectedObjectsComponent(2)  
Set swMathUtil = swApp.GetMathUtility  
  
vComp = swComp  
  
nPts(0) = 0#  
nPts(1) = 0#  
nPts(2) = 0#
```

```

vData = nPts
Set swOriginPt = swMathUtil.CreatePoint(vData)
nPts(0) = 1#
nPts(1) = 0#
nPts(2) = 0#
vData = nPts
Set swX_Axis = swMathUtil.CreateVector(vData)
' This is an incremental rotation,
' so angle is always the same.
Set swXform = swMathUtil.CreateTransformRotateAxis(swOriginPt, swX_Axis, 1# * RadPerDeg)
bRet = swDragOp.AddComponent(swComp(0), False)
Debug.Assert bRet
swDragOp.DynamicClearanceEnabled = False
' Axial rotation
swDragOp.TransformType = 1
' Solve by relaxation
swDragOp.DragMode = 2
bRet = swDragOp.BeginDrag
Debug.Assert bRet
For i = 0 To 500
    bRet = swDragOp.DragAsUI(swXform)
Next i
bRet = swDragOp.EndDrag
Debug.Assert bRet
End Sub

```

Rotate Assembly Component on Axis Using IDragOperator::DragAsUI Example (VB.NET)

This example shows how to rotate an assembly component about an assembly axis using IDragOperator::DragAsUI.

```
'-----
' Preconditions:
' 1. Assembly document is open.
' 2. Component in the assembly document is selected.
'
' Postconditions: Transform matrix is set.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Const PI As Double = 3.14159
    Const RadPerDeg As Double = PI / 180.0#

    ' DragOperator::TransformType
    ' Translation 0
    ' Transform is translation-only.
    '
    ' Axial rotation 1
    ' Transform is rotation-only.
    '
    ' General 2
    ' Transform can be translation, rotation, or both.
    '
    DragOperator::DragMode
    ' Minimum Move 0
    ' Move smallest number of geometries.
    '
    ' Maximum Move 1
    ' Move geometries rigidly if possible.
    '
    ' Relaxation 2
    ' Solve geometries using relaxation.
    '

Sub main()

    Dim swModel As ModelDoc2
    Dim swAssy As AssemblyDoc
    Dim swDragOp As DragOperator
    Dim swSelMgr As SelectionMgr
    Dim swComp(1) As Component2
    Dim vComp As Object
    Dim swXform As MathTransform
    Dim swMathUtil As MathUtility
    Dim swOriginPt As MathPoint
    Dim swX_Axis As MathVector
    Dim nPts(2) As Double
    Dim vData As Object
    Dim i As Integer
    Dim bRet As Boolean

    swModel = swApp.ActiveDoc
    swAssy = swModel
    swDragOp = swAssy.GetDragOperator
    swSelMgr = swModel.SelectionManager
    swComp(0) = swSelMgr.GetSelectedObjectsComponent(1)
    swComp(1) = swSelMgr.GetSelectedObjectsComponent(2)
```



```

swMathUtil = swApp.GetMathUtility

vComp = swComp

nPts(0) = 0.0#
nPts(1) = 0.0#
nPts(2) = 0.0#

vData = nPts

swOriginPt = swMathUtil.CreatePoint(vData)

nPts(0) = 1.0#
nPts(1) = 0.0#
nPts(2) = 0.0#
vData = nPts

swX_Axis = swMathUtil.CreateVector(vData)

' This is an incremental rotation,
' so angle is always the same.
swXform = swMathUtil.CreateTransformRotateAxis(swOriginPt, swX_Axis, 1.0# * RadPerDeg)

bRet = swDragOp.AddComponent(swComp(0), False)
Debug.Assert(bRet)

swDragOp.DynamicClearanceEnabled = False

' Axial rotation
swDragOp.TransformType = 1

' Solve by relaxation
swDragOp.DragMode = 2

bRet = swDragOp.BeginDrag
Debug.Assert(bRet)

For i = 0 To 500
    bRet = swDragOp.DragAsUI(swXform)
Next i

bRet = swDragOp.EndDrag
Debug.Assert(bRet)

End Sub

Public swApp As SldWorks

End Class

```

Rotate Assembly Component on Axis Using IDragOperator::Drag Example (C#)

This example shows how to rotate an assembly component about an assembly axis using IDragOperator::Drag.

NOTE: The code shows how to use a MathUtility object to directly create a transformation matrix (object) that represents rotation about a point and an axis, without having to know details of the OpenGL transformations.

```
//-----
// Preconditions: Verify that the specified assembly document exists.
//
// Postconditions:
// 1. Opens the specified assembly document, which is fully resolved.
// 2. Selects a floating component.
// 3. Watch the selected component in the graphics area rotate 90°
//    about the assembly's x axis.
//
// NOTE: Because this assembly is used elsewhere, do not save any changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace IDragOperatorCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        const double PI = 3.14159;
        const double RadPerDeg = PI / 180.0;

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            AssemblyDoc swAssy = default(AssemblyDoc);
            DragOperator swDragOp = default(DragOperator);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Component2 swComp = default(Component2);
            MathTransform swXform = default(MathTransform);
            MathUtility swMathUtil = default(MathUtility);
            MathPoint swOriginPt = default(MathPoint);
            MathVector swX_Axis = default(MathVector);
            string fileName = null;
            int errors = 0;
            int warnings = 0;
            bool status = false;
            double[] nPts = new double[3];
            object vData = null;
            System.DateTime nNow;
            int i = 0;
            bool bRet = false;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\api\\assem2.sldasm";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("Part3^Assem2-1@assem2", "COMPONENT", 0, 0, 0, false, 0, null, 0);
            swAssy = (AssemblyDoc)swModel;

            swDragOp = (DragOperator)swAssy.GetDragOperator();
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swComp = (Component2)swSelMgr.GetSelectedObjectsComponent2(1);
            swMathUtil = (MathUtility)swApp.GetMathUtility();

            nPts[0] = 0.0;
            nPts[1] = 0.0;
            nPts[2] = 0.0;
            vData = nPts;
            swOriginPt = (MathPoint)swMathUtil.CreatePoint(vData);

            nPts[0] = 1.0;
            nPts[1] = 0.0;
            nPts[2] = 0.0;
            vData = nPts;
            swX_Axis = (MathVector)swMathUtil.CreateVector(vData);

            // This is an incremental rotation,
            // so angle is always the same
            swXform = (MathTransform)swMathUtil.CreateTransformRotateAxis(swOriginPt, swX_Axis, 1.0 * RadPerDeg);
```

```

        bRet = swDragOp.AddComponent(swComp, false);

        swDragOp.CollisionDetectionEnabled = false;
        swDragOp.DynamicClearanceEnabled = false;

        // Axial rotation
        swDragOp.TransformType = 1;

        // Solve by relaxation
        swDragOp.DragMode = 2;

        bRet = swDragOp.BeginDrag();

        for (i = 0; i <= 90; i++)
        {
            // Returns false if drag fails
            bRet = swDragOp.Drag(swXform);
            // Wait for 0.01 secs
            nNow = System.DateTime.Now;
            while (System.DateTime.Now < nNow.AddSeconds(.01))
            {
                // Process event loop
                System.Windows.Forms.Application.DoEvents();
            }
        }

        bRet = swDragOp.EndDrag();
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
}

```

Rotate Assembly Component on Axis Using IDragOperator::Drag Example (VBA)

This example shows how to rotate an assembly component about an assembly axis using IDragOperator::Drag.

NOTE: The code shows how to use a MathUtility object to directly create a transformation matrix (object) that represents rotation about a point and an axis, without having to know details of the OpenGL transformations.

```
'-----  
' Preconditions: Verify that the specified assembly document exists.  
'  
' Postconditions:  
' 1. Opens the specified assembly document, which is fully resolved.  
' 2. Selects a floating component.  
' 3. Watch the selected component in the graphics area rotate  
'    90° about the assembly's x axis.  
'  
' NOTE: Because this assembly is used elsewhere, do not save any changes.  
'-----  
Option Explicit  
  
Const PI As Double = 3.14159  
Const RadPerDeg As Double = PI / 180#  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swModelDocExt As SldWorks.ModelDocExtension  
    Dim swAssy As SldWorks.AssemblyDoc  
    Dim swDragOp As SldWorks.DragOperator  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swComp As SldWorks.Component2  
    Dim swXform As SldWorks.MathTransform  
    Dim swMathUtil As SldWorks.MathUtility  
    Dim swOriginPt As SldWorks.MathPoint  
    Dim swX_Axis As SldWorks.MathVector  
    Dim fileName As String  
    Dim errors As Long  
    Dim warnings As Long  
    Dim status As Boolean  
    Dim nPts(2) As Double  
    Dim vData As Variant  
    Dim nNow As Single  
    Dim i As Long  
    Dim bRet As Boolean  
  
    Set swApp = Application.SldWorks  
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\assem2.sldasm"  
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY,  
swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)  
    Set swModelDocExt = swModel.Extension  
    status = swModelDocExt.SelectByID2("Part3^Assem2-1@assem2", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)  
    Set swAssy = swModel  
  
    Set swDragOp = swAssy.GetDragOperator  
    Set swSelMgr = swModel.SelectionManager  
    Set swComp = swSelMgr.GetSelectedObjectsComponent2(1)  
    Set swMathUtil = swApp.GetMathUtility  
  
    nPts(0) = 0#  
    nPts(1) = 0#  
    nPts(2) = 0#  
    vData = nPts  
    Set swOriginPt = swMathUtil.CreatePoint(vData)  
  
    nPts(0) = 1#  
    nPts(1) = 0#  
    nPts(2) = 0#  
    vData = nPts  
    Set swX_Axis = swMathUtil.CreateVector(vData)
```

```

' This is an incremental rotation,
' so angle is always the same
Set swXform = swMathUtil.CreateTransformRotateAxis(swOriginPt, swX_Axis, 1# * RadPerDeg)

bRet = swDragOp.AddComponent(swComp, False)

swDragOp.CollisionDetectionEnabled = False
swDragOp.DynamicClearanceEnabled = False

' Axial rotation
swDragOp.TransformType = 1

' Solve by relaxation
swDragOp.DragMode = 2

bRet = swDragOp.BeginDrag

For i = 0 To 90
    ' Returns false if drag fails
    bRet = swDragOp.Drag(swXform)
    ' Wait for 0.1 secs
    nNow = Timer
    While Timer < nNow + 0.1
        ' Process event loop
        DoEvents
    Wend
Next i

bRet = swDragOp.EndDrag

End Sub

```

Rotate Assembly Component on Axis Using IDragOperator::Drag Example (VB.NET)

This example shows how to rotate an assembly component about an assembly axis using IDragOperator::Drag.

NOTE: The code shows how to use a MathUtility object to directly create a transformation matrix (object) that represents rotation about a point and an axis, without having to know details of the OpenGL transformations.

```
'-----  
' Preconditions: Verify that the specified assembly document exists.  
'  
' Postconditions:  
' 1. Opens the specified assembly document, which is fully resolved.  
' 2. Selects a floating component.  
' 3. Watch the selected component in the graphics area rotate 90°  
'    about the assembly's x axis.  
'  
' NOTE: Because this assembly is used elsewhere, do not save any changes.  
'-----  
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System  
Imports System.Diagnostics  
Imports System.Windows.Forms.Application  
  
Partial Class SolidWorksMacro  
  
    Const PI As Double = 3.14159  
    Const RadPerDeg As Double = PI / 180.0#  
  
    Public Sub main()  
  
        Dim swModel As ModelDoc2  
        Dim swModelDocExt As ModelDocExtension  
        Dim swAssy As AssemblyDoc  
        Dim swDragOp As DragOperator  
        Dim swSelMgr As SelectionMgr  
        Dim swComp As Component2  
        Dim swXform As MathTransform  
        Dim swMathUtil As MathUtility  
        Dim swOriginPt As MathPoint  
        Dim swX_Axis As MathVector  
        Dim fileName As String  
        Dim errors As Integer  
        Dim warnings As Integer  
        Dim status As Boolean  
        Dim nPts(2) As Double  
        Dim vData As Object  
        Dim nNow As Single  
        Dim i As Integer  
        Dim bRet As Boolean  
  
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\assem2.sldasm"  
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Sil  
ent, "", errors, warnings)  
        swModelDocExt = swModel.Extension  
        status = swModelDocExt.SelectByID2("Part3^Assem2-1@assem2", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)  
        swAssy = swModel  
  
        swDragOp = swAssy.GetDragOperator  
        swSelMgr = swModel.SelectionManager  
        swComp = swSelMgr.GetSelectedObjectsComponent2(1)  
        swMathUtil = swApp.GetMathUtility  
  
        nPts(0) = 0.0#  
        nPts(1) = 0.0#  
        nPts(2) = 0.0#  
        vData = nPts  
        swOriginPt = swMathUtil.CreatePoint(vData)  
  
        nPts(0) = 1.0#  
        nPts(1) = 0.0#  
        nPts(2) = 0.0#  
        vData = nPts  
        swX_Axis = swMathUtil.CreateVector(vData)  
  
        ' This is an incremental rotation,  
        ' so angle is always the same  
        swXform = swMathUtil.CreateTransformRotateAxis(swOriginPt, swX_Axis, 1.0# * RadPerDeg)
```

```

bRet = swDragOp.AddComponent(swComp, False)

swDragOp.CollisionDetectionEnabled = False
swDragOp.DynamicClearanceEnabled = False

' Axial rotation
swDragOp.TransformType = 1

' Solve by relaxation
swDragOp.DragMode = 2

bRet = swDragOp.BeginDrag

For i = 0 To 90
    ' Returns false if drag fails
    bRet = swDragOp.Drag(swXform)
    ' Wait for 0.1 secs
    nNow = Timer
    While Timer < nNow + 0.1
        ' Process event loop
        DoEvents()
    End While
Next i

bRet = swDragOp.EndDrag

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Rotate Drawing View 45 Degrees Example (C#)

This example shows how to rotate the selected drawing view 45°.

```
//-----  
// Preconditions: Verify that the specified file to open exists.  
//  
// Postconditions: Rotates the selected drawing view 45°.  
//-----  
  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System;  
  
namespace DrawingViewRotateCSharp.csproj  
{  
  
    partial class SolidWorksMacro  
    {  
  
        public void Main()  
        {  
  
            ModelDoc2 swModel = default(ModelDoc2);  
            ModelDocExtension swModelDocExt = default(ModelDocExtension);  
            DrawingDoc swDrawing = default(DrawingDoc);  
            bool status = false;  
            int errors = 0;  
            int warnings = 0;  
  
            swModel =  
(ModelDoc2)swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS  
2018\\samples\\tutorial\\driveworkspress\\mobile gantry.slddrw",  
(int)swDocumentTypes_e.swDocDRAWING, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref  
errors, ref warnings);  
            swModelDocExt = (ModelDocExtension)swModel.Extension;  
            swModel.ViewZoomtofit2();  
            swDrawing = (DrawingDoc)swModel;  
  
            status = swDrawing.ActivateView("Drawing View4");  
            status = swModelDocExt.SelectByID2("Drawing View4", "DRAWINGVIEW", 0.1122300799499,  
0.1471819585104, 0, false, 0, null, 0);  
  
            //Convert degrees to radians, the default system unit  
            // 1 radian = 180°/p = 57.295779513° or approximately 57.3°  
            status = swDrawing.DrawingViewRotate(45 / 57.3);  
        }  
  
        /// <summary>  
        /// The SldWorks swApp variable is pre-assigned for you.  
        /// </summary>  
        public SldWorks swApp;  
    }  
}
```


Rotate Drawing Views 45 Degrees Example (VBA)

This example shows how to rotate the selected drawing view 45°.

```
'-----  
' Preconditions: Verify that the specified file to open exists.  
'  
' Postconditions: Rotates the selected drawing view 45°.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swDrawing As SldWorks.DrawingDoc  
Dim status As Boolean  
Dim errors As Long  
Dim warnings As Long
```

Sub main()

```
    Set swApp = Application.SldWorks  
    Set swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS  
2018\samples\tutorial\driveworkspress\mobile gantry.slddrw", swDocDRAWING, swOpenDocOptions_Silent,  
    "", errors, warnings)  
    Set swModelDocExt = swModel.Extension  
    swModel.ViewZoomtofit2  
    Set swDrawing = swModel  
  
    status = swDrawing.ActivateView("Drawing View4")  
    status = swModelDocExt.SelectByID2("Drawing View4", "DRAWINGVIEW", 0.1122300799499, 0.1471819585104, 0,  
    False, 0, Nothing, 0)  
  
    'Convert degrees to radians, the default system unit  
    ' 1 radian = 180°/p = 57.295779513° or approximately 57.3°  
    status = swDrawing.DrawingViewRotate(45 / 57.3)
```

End Sub

Rotate Drawing View 45 Degrees Example (VB.NET)

This example shows how to rotate the selected drawing view 45°.

```
'-----  
' Preconditions: Verify that the specified file to open exists.  
'  
' Postconditions: Rotates the selected drawing view 45°.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Public Sub main()
```

```
        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swDrawing As DrawingDoc
        Dim status As Boolean
        Dim errors As Long
        Dim warnings As Long
```

```
        swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\driveworksxpess\mobile gantry.slddrw", swDocumentTypes_e.swDocDRAWING,
swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModelDocExt = swModel.Extension
        swModel.ViewZoomtofit2()
        swDrawing = swModel
```

```
        status = swDrawing.ActivateView("Drawing View4")
        status = swModelDocExt.SelectByID2("Drawing View4", "DRAWINGVIEW", 0.1122300799499,
0.1471819585104, 0, False, 0, Nothing, 0)

        'Convert degrees to radians, the default system unit
        ' 1 radian = 180°/p = 57.295779513° or approximately 57.3°
        status = swDrawing.DrawingViewRotate(45 / 57.3)
```

```
    End Sub
```

```
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
```

```
End Class
```

Rotate Model Example (C#)

This example shows how to rotate a model in the graphics area.

```
//-----
// Preconditions:
// 1. Verify that the specified part template exists.
// 2. Watch the graphics while the macro runs.
//
// Postconditions:
// 1. Creates a new part document.
// 2. Inserts and extrudes a rectangular sketch.
// 3. Rotates the sketch multiple times.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace RotateModel_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;
        Feature myFeature;
        object vSkLines;
        bool boolstatus;

        public void Main()
        {
            Part = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SOLIDWORKS\\SOLIDWORKS
2016\\templates\\Part.prtdot", 0, 0, 0);
            boolstatus = Part.Extension.SelectByID2("Top Plane", "PLANE", -0.0567254111166863,
0.00753958008310182, 0.0248109468921342, false, 0, null, 0);
            Part.SketchManager.InsertSketch(true);
            vSkLines = Part.SketchManager.CreateCornerRectangle(-0.0493169981371904, 0.0173783707721528, 0,
0.0558925978888158, -0.0455595125648331, 0);
            Part.ShowNamedView2("Trimetric", 8);
            boolstatus = Part.Extension.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, false, 0, null, 0);
            boolstatus = Part.Extension.SelectByID2("Line1", "SKETCHSEGMENT", 0, 0, 0, true, 0, null, 0);
            boolstatus = Part.Extension.SelectByID2("Line4", "SKETCHSEGMENT", 0, 0, 0, true, 0, null, 0);
            boolstatus = Part.Extension.SelectByID2("Line3", "SKETCHSEGMENT", 0, 0, 0, true, 0, null, 0);
            myFeature = Part.FeatureManager.FeatureExtrusion2(true, false, false, 0, 0, 0.016256, 0.00254, false,
false, false, false, 0.0174532925199433, 0.0174532925199433, false, false, false, true, true, true, 0, 0,
false);

            ((SelectionMgr)(Part.SelectionManager)).EnableContourSelection = false;

            Part.ViewRotate();
            Part.ViewRotateminusx();
            Part.ViewRotateminusy();
            Part.ViewRotateminusz();
            Part.ViewRotateplusx();
            Part.ViewRotateplusy();
            Part.ViewRotateplusz();
            Part.ViewRotXMinusNinety();
            Part.ViewRotXPlusNinety();
            Part.ViewRotYMinusNinety();
            Part.ViewRotYPlusNinety();
        }

        public SldWorks swApp;
    }
}
```

Rotate Model Example (VBA)

This example shows how to rotate a model in the graphics area.

```
'-----  
' Preconditions:  
' 1. Verify that the specified part template exists.  
' 2. Watch the graphics while the macro runs.  
'  
' Postconditions:  
' 1. Creates a new part document.  
' 2. Inserts and extrudes a rectangular sketch.  
' 3. Rotates the sketch multiple times.  
'-----  
Dim swApp As SldWorks.SldWorks  
  
Dim Part As SldWorks.ModelDoc2  
  
Dim myFeature As SldWorks.Feature  
  
Dim vSkLines As Variant  
  
Dim boolstatus As Boolean  
  
Option Explicit  
  
Sub main()  
    Set swApp = Application.SldWorks  
  
    Set Part = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS  
2012\templates\Part.prtdot", 0, 0, 0)  
  
    boolstatus = Part.Extension.SelectByID2("Top Plane", "PLANE", -5.67254111166863E-02,  
7.53958008310182E-03, 2.48109468921342E-02, False, 0, Nothing, 0)  
    Part.SketchManager.InsertSketch True  
  
    vSkLines = Part.SketchManager.CreateCornerRectangle(-4.93169981371904E-02, 1.73783707721528E-  
02, 0, 5.58925978888158E-02, -4.55595125648331E-02, 0)  
    Part.ShowNamedView2 "Trimetric", 8  
  
    boolstatus = Part.Extension.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing,  
0)  
    boolstatus = Part.Extension.SelectByID2("Line1", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing,  
0)  
    boolstatus = Part.Extension.SelectByID2("Line4", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing,  
0)  
    boolstatus = Part.Extension.SelectByID2("Line3", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing,  
0)  
  
    Set myFeature = Part.FeatureManager.FeatureExtrusion2(True, False, False, 0, 0, 0.016256,  
0.00254, False, False, False, False, 1.74532925199433E-02, 1.74532925199433E-02, False, False,  
False, False, True, True, True, 0, 0, False)  
    Part.SelectionManager.EnableContourSelection = False  
  
    Part.ViewRotate  
    Part.ViewRotateminusx  
    Part.ViewRotateminusy  
    Part.ViewRotateminusz  
    Part.ViewRotateplusx  
    Part.ViewRotateplusy  
    Part.ViewRotateplusz  
    Part.ViewRotXMinusNinety  
    Part.ViewRotXPlusNinety
```

Part.**ViewRotYMinusNinety**
Part.**ViewRotYPlusNinety**

End Sub

Rotate Model Example (VB.NET)

This example shows how to rotate a model in the graphics area.

```
'-----  
' Preconditions:  
' 1. Verify that the specified part template exists.  
' 2. Watch the graphics while the macro runs.  
'-----  
' Postconditions:  
' 1. Creates a new part document.  
' 2. Inserts and extrudes a rectangular sketch.  
' 3. Rotates the sketch multiple times.  
'-----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System.Runtime.InteropServices
```

```
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Dim Part As ModelDoc2
```

```
    Dim myFeature As Feature
```

```
    Dim vSkLines As Object
```

```
    Dim boolstatus As Boolean
```

```
    Sub main()
```

```
        Part = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS  
2016\templates\Part.prtdot", 0, 0, 0)
```

```
        boolstatus = Part.Extension.SelectByID("Top  
Plane", "PLANE", -0.0567254111166863,  
0.00753958008310182, 0.0248109468921342, False, 0, Nothing, 0)
```

```
        Part.SketchManager.InsertSketch(True)
```

```
        vSkLines = Part.SketchManager.CreateCornerRectangle(-0.0493169981371904,  
0.0173783707721528, 0, 0.0558925978888158, -0.0455595125648331, 0)
```

```
        Part.ShowNamedView2("Trimetric",  
8)
```

```
        boolstatus = Part.Extension.SelectByID("Line2", "SKETCHSEGMENT",  
0, 0, 0, False,  
0, Nothing,  
0)
```

```
        boolstatus = Part.Extension.SelectByID("Line1", "SKETCHSEGMENT",  
0, 0, 0, True,  
0, Nothing,  
0)
```

```
        boolstatus = Part.Extension.SelectByID("Line4", "SKETCHSEGMENT",  
0, 0, 0, True,  
0, Nothing,
```

```

0)

    boolstatus = Part.Extension.SelectByID("Line3", "SKETCHSEGMENT",
0, 0, 0, True,
0, Nothing,
0)

    myFeature = Part.FeatureManager.FeatureExtrusion2(True, False, False, 0,
0, 0.016256, 0.00254, False, False, False, False,
0.0174532925199433, 0.0174532925199433, False, False, False, False, True, True, True, 0, 0, False)

    Part.SelectionManager.EnableContourSelection = False

    Part.ViewRotate()

    Part.ViewRotateminusx()

    Part.ViewRotateminusy()

    Part.ViewRotateminusz()

    Part.ViewRotateplusx()

    Part.ViewRotateplusy()

    Part.ViewRotateplusz()

    Part.ViewRotXMinusNinety()

    Part.ViewRotXPlusNinety()

    Part.ViewRotYMinusNinety()

    Part.ViewRotYPlusNinety()

End Sub

Public swApp As SldWorks

End Class

```

Rotate Move Face Feature Example (C#)

This example shows how to rotate (draft) a Move Face feature by changing the XYZ origin and rotation angles.

```
//-----  
// Preconditions:  
// 1. Open a part document that contains a Move Face feature named Move Face1.  
// 2. Open the Immediate window.  
//  
// Postconditions:  
// 1. Rotates (drafts) the Move Face feature.  
// 2. Examine the Immediate window.  
//-----  
  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System;  
using System.Diagnostics;  
namespace RotateMoveFaceFeature_CSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
        public void Main()  
        {  
            ModelDoc2 swModel = default(ModelDoc2);  
            ModelDocExtension swModelDocExt = default(ModelDocExtension);  
            SelectionMgr swSelMgr = default(SelectionMgr);  
            Feature swFeat = default(Feature);  
            MoveFaceFeatureData swMoveFaceFeatData = default(MoveFaceFeatureData);  
            double[] varPara = null;  
            double[] newPara = new double[6];  
            double[] newVarPara = null;  
            bool boolstatus = false;  
            double PI = 0;  
            long i = 0;  
            // Set PI  
            PI = 4 * System.Math.Atan(1);  
            swModel = (ModelDoc2)swApp.ActiveDoc;  
            swSelMgr = (SelectionMgr)swModel.SelectionManager;  
            swModelDocExt = swModel.Extension;  
            // Select, get, and access Move Face feature  
            boolstatus = swModelDocExt.SelectByID2("Move Face1", "BODYFEATURE", 0, 0, 0, false, 0,  
null, 0);  
  
            swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);  
            swMoveFaceFeatData = (MoveFaceFeatureData)swFeat.GetDefinition();  
            swMoveFaceFeatData.AccessSelections(swModel, null);  
            // Get current XYZ origin and rotation angles of Move Face feature
```



```

        Debug.Print("Before rotating Move Face feature...");
        // 1 radian = 180°/p = 57.295779513° or approximately 57.3°
        Debug.Print(" Draft angle of Move Face feature: " + swMoveFaceFeatData.Angle * 57.3 +
" degrees");
        Debug.Print(" XYZ origin (first 3) and rotation angles (last 3)");
        varPara = (double[])swMoveFaceFeatData.TriadRotationParameters;
        for (i = varPara.GetLowerBound(0); i <= varPara.GetUpperBound(0); i++)
        {
            Debug.Print(" " + (varPara[i]));
        }
        // New XYZ location and rotation angle values
        newPara[0] = 0.0;
        newPara[1] = 0.0;
        newPara[2] = 0.0;
        newPara[3] = 2 * PI / 180;
        // Convert radians to degrees
        newPara[4] = 2 * PI / 180;
        // Convert radians to degrees
        newPara[5] = 0.0;
        newVarPara = newPara;
        // Rotate the Move Face feature
        swMoveFaceFeatData.TriadRotationParameters = newVarPara;
        Debug.Print(" ");
        Debug.Print("After rotating Move Face feature...");
        Debug.Print(" Draft angle of Move Face feature: " + swMoveFaceFeatData.Angle * 57.3 +
" degrees");
        Debug.Print(" XYZ origin (first 3) and rotation angles (last 3)");
        newVarPara = (double[])swMoveFaceFeatData.TriadRotationParameters;
        for (i = newVarPara.GetLowerBound(0); i <= newVarPara.GetUpperBound(0); i++)
        {
            Debug.Print(" " + (newVarPara[i]));
        }
        // Modify the Move Face feature
        swFeat.ModifyDefinition(swMoveFaceFeatData, swModel, null);
    }

    public SldWorks swApp;
}
}

```

Rotate Move Face Feature Example (VB.NET)

This example shows how to rotate (draft) a Move Face feature by changing the XYZ origin and rotation angles.

```
'-----  
' Preconditions:  
' 1. Open a part document that contains a Move Face feature named Move Face1.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Rotates (drafts) the Move Face feature.  
' 2. Examine the Immediate window.  
'-----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System
```

```
Imports System.Diagnostics
```

```
Imports System.Math
```

```
Partial Class SolidWorksMacro
```

```
    Public Sub main()
```

```
        Dim swModel As ModelDoc2
```

```
        Dim swModelDocExt As ModelDocExtension
```

```
        Dim swSelMgr As SelectionMgr
```

```
        Dim swFeat As Feature
```

```
        Dim swMoveFaceFeatData As MoveFaceFeatureData
```

```
        Dim varPara As Object
```

```
        Dim newPara(5) As Double
```

```
        Dim newVarPara As Object
```

```
        Dim boolstatus As Boolean
```

```
        Dim PI As Double
```

```
        Dim i As Long
```

```
        ' Set PI
```

```
        PI = 4 * Atan(1)
```

```
        swModel = swApp.ActiveDoc
```

```
        swSelMgr = swModel.SelectionManager
```

```
        swModelDocExt = swModel.Extension
```

```
        ' Select, get, and access Move Face feature
```

```
        boolstatus = swModelDocExt.SelectByID2("Move Face1", "BODYFEATURE", 0, 0, 0, False, 0,  
Nothing, 0)
```

```
        swFeat = swSelMgr.GetSelectedObject6(1, -1)
```

```
        swMoveFaceFeatData = swFeat.GetDefinition
```

```

swMoveFaceFeatData.AccessSelections(swModel, Nothing)

' Get current XYZ location and rotation angles of Move Face feature
Debug.Print("Before rotating Move Face feature...")
' 1 radian = 180°/p = 57.295779513° or approximately 57.3°
Debug.Print(" Draft angle of Move Face feature: " & swMoveFaceFeatData.Angle * 57.3 & "
degrees")
Debug.Print(" XYZ origin (first 3) and rotation angles (last 3)")
varPara = swMoveFaceFeatData.TriadRotationParameters
For i = LBound(varPara) To UBound(varPara)
    Debug.Print(" " & (varPara(i)))
Next i

' New XYZ location and rotation angle values
newPara(0) = 0.0#
newPara(1) = 0.0#
newPara(2) = 0.0#
newPara(3) = 2 * PI / 180 ' Convert radians to degrees
newPara(4) = 2 * PI / 180 ' Convert radians to degrees
newPara(5) = 0.0#
newVarPara = newPara

' Rotate the MoveFace feature
swMoveFaceFeatData.TriadRotationParameters = newVarPara
Debug.Print(" ")
Debug.Print("After rotating Move Face feature...")
Debug.Print(" Draft angle of Move Face feature: " & swMoveFaceFeatData.Angle * 57.3 & "
degrees")
Debug.Print(" XYZ origin (first 3) and rotation angles (last 3)")
newVarPara = swMoveFaceFeatData.TriadRotationParameters
For i = LBound(newVarPara) To UBound(newVarPara)
    Debug.Print(" " & (newVarPara(i)))
Next i

' Modify the Move Face feature
swFeat.ModifyDefinition(swMoveFaceFeatData, swModel, Nothing)

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Rotate Move Face Feature Example (VBA)

This example shows how to rotate (draft) a Move Face feature by changing its XYZ location and rotation angles.

```
'-----  
' Preconditions:  
' 1. Open a part document that contains a Move Face feature named Move Face1.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Rotates (drafts) the Move Face feature.  
' 2. Examine the Immediate window.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swFeat As SldWorks.Feature  
Dim swMoveFaceFeatData As SldWorks.MoveFaceFeatureData  
Dim varPara As Variant  
Dim newPara(5) As Double  
Dim newVarPara As Variant  
Dim boolstatus As Boolean  
Dim PI As Double  
Dim i As Long  
  
Sub main()  
  
    ' Set PI  
    PI = 4 * Atn(1)  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swModelDocExt = swModel.Extension  
  
    ' Select, get, and access Move Face feature  
    boolstatus = swModelDocExt.SelectByID2("Move Face1", "BODYFEATURE", 0, 0, 0, False, 0,  
    Nothing, 0)  
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)  
    Set swMoveFaceFeatData = swFeat.GetDefinition  
    swMoveFaceFeatData.AccessSelections swModel, Nothing  
  
    ' Get current XYZ origin and rotation angles of Move Face feature  
    Debug.Print "Before rotating Move Face feature..."
```

```

' 1 radian = 180°/p = 57.295779513° or approximately 57.3°
Debug.Print " Draft angle of Move Face feature: " & swMoveFaceFeatData.Angle * 57.3 & "
degrees"
Debug.Print " XYZ origin (first 3) and rotation angles (last 3)"
varPara = swMoveFaceFeatData.TriadRotationParameters
For i = LBound(varPara) To UBound(varPara)
    Debug.Print " " & (varPara(i))
Next i

' New XYZ location and rotation angles for Move Face feature
newPara(0) = 0#
newPara(1) = 0#
newPara(2) = 0#
newPara(3) = 2 * PI / 180 ' Convert radians to degrees
newPara(4) = 2 * PI / 180 ' Convert radians to degrees
newPara(5) = 0#
newVarPara = newPara

' Rotate the Move Face feature
swMoveFaceFeatData.TriadRotationParameters = newVarPara
Debug.Print " "
Debug.Print "After rotating Move Face feature..."
Debug.Print " Draft angle of Move Face feature: " & swMoveFaceFeatData.Angle * 57.3 & "
degrees"
Debug.Print " XYZ origin (first 3) and rotation angles (last 3)"
newVarPara = swMoveFaceFeatData.TriadRotationParameters
For i = LBound(newVarPara) To UBound(newVarPara)
    Debug.Print " " & (newVarPara(i))
Next i

' Modify the Move Face feature
swFeat.ModifyDefinition swMoveFaceFeatData, swModel, Nothing

```

End Sub

Rotate, Scale, and Copy Sketch Example (C#)

This example shows how to rotate a sketch; scale, copy, and rotate selected entities of the sketch; and scale the entire sketch.

```
//-----
// Preconditions: Verify that the specified part document
// template exists.
//
// Postconditions:
// Step through the macro and observe:
// 1. Creates a sketch of a rectangle.
// 2. Rotates the sketch about the specified points.
// 3. Makes a copy of the selected sketch lines and
//    scales them by a factor of 2.
// 4. Rotates the selected line.
// 5. Zooms to fit the sketch.
// 6. Scales the sketch by a factor of 3.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace SketchRotateScaleCopyCSharp.csproj
{
    public partial class SolidWorksMacro
    {

        public void Main()
        {

            ModelDoc2 swModel = default(ModelDoc2);
            SketchManager swSketchMgr = default(SketchManager);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            object sketchLines = null;
            bool status = false;

            // Open a new part document
            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SOLIDWORKS\\SOLIDWORKS 2015\\templates\\Part.p
rtdot", 0, 0, 0);
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            // Create sketch of rectangle on the Front plane
            swSketchMgr = (SketchManager)swModel.SketchManager;
            status = swModelDocExt.SelectByID2("Front Plane", "PLANE", -0.0212975109505464, 0.121561074451165, 0.10
0935818984055, false, 0, null, 0);
            swModel.ClearSelection2(true);
            swSketchMgr.InsertSketch(true);
            swModel.ClearSelection2(true);
            sketchLines = (object)swSketchMgr.CreateCornerRectangle(0, 0, 0, -0.0822154876580373, 0.063635716435882
, 0);

            swModel.ClearSelection2(true);

            // Rotate the sketch about the specified point
            swModel.SketchModifyRotate(1, 1, 1);

            swModel.ClearSelection2(true);

            // Make a copy of the selected lines and scale them by a factor of 2
            status = swModelDocExt.SelectByID2("Line3", "SKETCHSEGMENT", -0.0630770706086407, 0.0172671115438625, 0
.0215538897292735, true, 0, null, 0);
            status = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", -0.0360136822942443, 0.0250170683049161, 0
.00200770232274633, true, 0, null, 0);
            status = swModelDocExt.SelectByID2("Line1", "SKETCHSEGMENT", -0.00735948431462766, -0.0130061570540028,
0.0127196907180518, true, 0, null, 0);
            status = swModelDocExt.SelectByID2("Line4", "SKETCHSEGMENT", -0.0501900457103943, -0.0224514168565368,
0.0417377643321936, true, 0, null, 0);
            swModelDocExt.ScaleOrCopy(true, 2, 0, 0.063635716435882, 0, 2);

            // Rotate selected Line3
            status = swModel.DeSelectByID("Line2", "SKETCHSEGMENT", 0.00159286151716137, 0.0438212483979034, 0.0020
0770232274633);
```

```

        status = swModel.DeSelectByID("Line1", "SKETCHSEGMENT", -0.0149206501299916, -0.00083446413285288, 0.01
27196907180518);
        status = swModel.DeSelectByID("Line4", "SKETCHSEGMENT", -0.046010013281556, 0.0301029148938852, 0.04173
77643321936);
        swModelDocExt.RotateOrCopy(false, 2, false, -0.164430975316075, 0.063635716435882, 0, 0, 0, 1, 0.785398
16339745);
        swModel.ClearSelection2(true);

        // Zoom to fit
        swModel.ViewZoomtofit2();

        // Scale the sketch by a factor of 3
        status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, false, 0, null, 0);
        status = swModel.SketchModifyScale(3);

    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
}

```

Rotate, Scale, and Copy Sketch Example (VBA)

This example shows how to rotate a sketch; scale, copy, and rotate selected entities of the sketch; and scale the entire sketch.

```
'-----  
' Preconditions: Verify that the specified part document template exists.  
'-----  
' Postconditions:  
' Step through the macro and observe:  
' 1. Creates a sketch of a rectangle.  
' 2. Rotates the sketch about the specified points.  
' 3. Makes a copy of the selected sketch lines and  
'    scales them by a factor of 2.  
' 4. Rotates the selected line.  
' 5. Zooms to fit the sketch.  
' 6. Scales the sketch by a factor of 3.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swSelectionMgr As SldWorks.SelectionMgr  
Dim sketchLines As Variant  
Dim status As Boolean  
  
Sub main()  
  
Set swApp = Application.SldWorks  
  
' Open a new part document  
Set swModel = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2015\templates\Part.prtdot", 0, 0, 0)  
Set swModelDocExt = swModel.Extension  
  
' Create sketch of rectangle on the Front plane  
Set swSketchMgr = swModel.SketchManager  
status = swModelDocExt.SelectByID("Front Plane", "PLANE", -2.12975109505464E-02, 0.121561074451165,  
0.100935818984055, False, 0, Nothing, 0)  
swModel.ClearSelection2 True  
swSketchMgr.InsertSketch True  
swModel.ClearSelection2 True  
sketchLines = swSketchMgr.CreateCornerRectangle(0, 0, 0, -8.22154876580373E-02, 0.063635716435882, 0)  
swModel.ClearSelection2 True  
  
' Rotate the sketch about the specified point  
swModel.SketchModifyRotate 1, 1, 1  
  
swModel.ClearSelection2 True  
  
' Make a copy of the selected lines and scale them by a factor of 2  
status = swModelDocExt.SelectByID("Line3", "SKETCHSEGMENT", -6.30770706086407E-02, 1.72671115438625E-02,  
2.15538897292735E-02, True, 0, Nothing, 0)  
status = swModelDocExt.SelectByID("Line2", "SKETCHSEGMENT", -3.60136822942443E-02, 2.50170683049161E-02,  
2.00770232274633E-03, True, 0, Nothing, 0)  
status = swModelDocExt.SelectByID("Line1", "SKETCHSEGMENT", -7.35948431462766E-03, -1.30061570540028E-02,  
1.27196907180518E-02, True, 0, Nothing, 0)  
status = swModelDocExt.SelectByID("Line4", "SKETCHSEGMENT", -5.01900457103943E-02, -2.24514168565368E-02,  
4.17377643321936E-02, True, 0, Nothing, 0)  
swModelDocExt.ScaleOrCopy True, 2, 0, 0.063635716435882, 0, 2  
  
' Rotate selected Line3  
status = swModel.DeSelectByID("Line2", "SKETCHSEGMENT", 1.59286151716137E-03, 4.38212483979034E-02,  
2.00770232274633E-03)  
status = swModel.DeSelectByID("Line1", "SKETCHSEGMENT", -1.49206501299916E-02, -8.3446413285288E-04,  
1.27196907180518E-02)  
status = swModel.DeSelectByID("Line4", "SKETCHSEGMENT", -0.046010013281556, 3.01029148938852E-02,  
4.17377643321936E-02)
```



```
swModelDocExt.RotateOrCopy False, 2, False, -0.164430975316075, 0.063635716435882, 0, 0, 0, 1,  
0.78539816339745
```

```
swModel.ClearSelection2 True
```

```
' Zoom to fit
```

```
swModel.ViewZoomtofit2
```

```
' Scale the sketch by a factor of 3
```

```
status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
```

```
status = swModel.SketchModifyScale(3)
```

```
End Sub
```

Rotate, Scale, and Copy Sketch Example (VB.NET)

This example shows how to rotate a sketch; scale, copy, and rotate selected entities of the sketch; and scale the entire sketch.

```

'-----
' Preconditions: Verify that the specified part document
' template exists.
'
' Postconditions:
' Step through the macro and observe:
' 1. Creates a sketch of a rectangle.
' 2. Rotates the sketch about the specified points.
' 3. Makes a copy of the selected sketch lines and
'   scales them by a factor of 2.
' 4. Rotates the selected line.
' 5. Zooms to fit the sketch.
' 6. Scales the sketch by a factor of 3.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swSketchMgr As SketchManager
        Dim swModelDocExt As ModelDocExtension
        Dim sketchLines As Object
        Dim status As Boolean

        ' Open a new part document
        swModel = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2015\templates\Part.prtdot", 0, 0, 0)
        swModelDocExt = swModel.Extension

        ' Create sketch of rectangle on the Front plane
        swSketchMgr = swModel.SketchManager
        status = swModelDocExt.SelectByID2("Front Plane", "PLANE", -0.0212975109505464, 0.121561074451165, 0.100935
818984055, False, 0, Nothing, 0)
        swModel.ClearSelection2(True)
        swSketchMgr.InsertSketch(True)
        swModel.ClearSelection2(True)
        sketchLines = swSketchMgr.CreateCornerRectangle(0, 0, 0, -0.0822154876580373, 0.063635716435882, 0)
        swModel.ClearSelection2(True)

        ' Rotate the sketch about the specified point
        swModel.SketchModifyRotate(1, 1, 1)

        swModel.ClearSelection2(True)

        ' Make a copy of the selected lines and scale them by a factor of 2
        status = swModelDocExt.SelectByID2("Line3", "SKETCHSEGMENT", -0.0630770706086407, 0.0172671115438625, 0.021
5538897292735, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", -0.0360136822942443, 0.0250170683049161, 0.002
00770232274633, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("Line1", "SKETCHSEGMENT", -0.00735948431462766, -0.0130061570540028, 0.0
127196907180518, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("Line4", "SKETCHSEGMENT", -0.0501900457103943, -0.0224514168565368, 0.04
17377643321936, True, 0, Nothing, 0)
        swModelDocExt.ScaleOrCopy(True, 2, 0, 0.063635716435882, 0, 2)

        ' Rotate selected Line3
        status = swModel.DeSelectByID("Line2", "SKETCHSEGMENT", 0.00159286151716137, 0.0438212483979034, 0.00200770
232274633)
        status = swModel.DeSelectByID("Line1", "SKETCHSEGMENT", -0.0149206501299916, -0.00083446413285288, 0.012719
6907180518)
        status = swModel.DeSelectByID("Line4", "SKETCHSEGMENT", -0.046010013281556, 0.0301029148938852, 0.041737764
3321936)
        swModelDocExt.RotateOrCopy(False, 2, False, -0.164430975316075, 0.063635716435882, 0, 0, 0, 1, 0.7853981633
9745)
        swModel.ClearSelection2(True)

        ' Zoom to fit
        swModel.ViewZoomtofit2()

        ' Scale the sketch by a factor of 3
    
```

```
status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
status = swModel.SketchModifyScale(3)
```

```
End Sub
```

```
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
```

```
End Class
```

Run Interference Detection Example (C#)

This example shows how to run interference detection and observe the interferences in an assembly.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\advdrawings\ Bowl and chute.sldasm.
// 2. Ensure c:\temp exists.
// 3. Open the Immediate window.
//
// Postconditions:
// 1. Opens the Interference Detection pane.
// 2. Gets interferences.
// 3. Gets interfering components and transforms.
// 4. Gets interference information.
// 5. In the Results list on the Interference Detection pane,
//    click on each interference and observe it in the graphics area (red).
// 6. To close the Interference Detection pane, right-click the graphics area
//    and select Cancel.
// 7. Inspect the Immediate window and c:\temp\IntDetReport.xlsx.
//
// NOTE: Because the assembly is used elsewhere, do not save changes.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace Interference_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModelDoc = default(ModelDoc2);
            AssemblyDoc swAssemblyDoc = default(AssemblyDoc);
            InterferenceDetectionMgr pIntMgr = default(InterferenceDetectionMgr);
            object[] vInts = null;
            long i = 0;
            long j = 0;
            IInterference interference = default(IInterference);
            object vIntComps = null;
            object[] vComps = null;
            Component2 comp = default(Component2);
            double vol = 0;
            object vTrans = null;
            bool ret = false;

            swModelDoc = (
                ModelDoc2)swApp.ActiveDoc;
            swAssemblyDoc = (AssemblyDoc)swModelDoc;

            //Open the Interference Detection pane
            swAssemblyDoc.ToolsCheckInterference();
            pIntMgr = swAssemblyDoc.InterferenceDetectionManager;

            //Specify the interference detection settings and options
            pIntMgr.TreatCoincidenceAsInterference = false;
        }
    }
}
```

```

pIntMgr.TreatSubAssembliesAsComponents = false;
pIntMgr.IncludeMultibodyPartInterferences = true;
pIntMgr.MakeInterferingPartsTransparent = true;
pIntMgr.CreateFastenersFolder = false;
pIntMgr.IgnoreHiddenBodies = false;
pIntMgr.ShowIgnoredInterferences = false;
pIntMgr.UseTransform = false;

//Specify how to display non-interfering components
pIntMgr.NonInterferingComponentDisplay =
(int)swNonInterferingComponentDisplay_e.swNonInterferingComponentDisplay_Wireframe;

//Run interference detection
vInts = (object[])pIntMgr.GetInterferences();
Debug.Print("Number of interferences: " +
pIntMgr.GetInterferenceCount());

//Get interfering components and transforms
ret = pIntMgr.GetComponentsAndTransforms(out vIntComps, out vTrans);

//Get interference information
for (i = 0; i <= vInts.GetUpperBound(0); i++)
{
    Debug.Print("Interference " + (i + 1));
    interference = (IInterference)vInts[i];
    Debug.Print("    Number of components in this interference: "
+ interference.GetComponentCount());
    vComps = (object[])interference.Components;
    for (j = 0; j <= vComps.GetUpperBound(0); j++)
    {
        comp = (Component2)vComps[j];
        Debug.Print("    " + comp.Name2);
    }
    vol = interference.Volume;
    Debug.Print("    Interference volume is " + (vol *
1000000000) + " mm^3");
}

ret = pIntMgr.ExportResults("c:\\temp\\IntDetReport.xlsx", true);

//Stop interference detection and close Inteference Detection pane
//pIntMgr.Done();
}

public SldWorks swApp;
}
}

```

Run Interference Detection Example (VBA)

This example shows how to run interference detection and observe the interferences in an assembly.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\advdrawings\ Bowl and chute.sldasm.  
' 2. Ensure c:\temp exists.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Opens the Interference Detection pane.  
' 2. Gets interferences.  
' 3. Gets interfering components and transforms.  
' 4. Gets interference information.  
' 5. In the Results list on the Interference Detection pane,  
'    click on each interference and observe it in the graphics area (red).  
' 6. To close the Interference Detection pane, right-click the graphics area  
'    and select Cancel.  
' 7. Inspect the Immediate window and c:\temp\IntDetReport.xlsx.  
'  
' NOTE: Because the assembly is used elsewhere, do not save changes.  
'-----
```

Option Explicit

```
Sub main()  
    Dim swApp As SldWorks.SldWorks  
    Dim swModelDoc As SldWorks.ModelDoc2  
    Dim swAssemblyDoc As SldWorks.AssemblyDoc  
    Dim pIntMgr As SldWorks.InterferenceDetectionMgr  
    Dim vInts As Variant  
    Dim i As Long  
    Dim j As Long  
    Dim interference As IInterference  
    Dim vComps As Variant  
    Dim comp As Component2  
    Dim vol As Double  
    Dim vTrans As Variant  
    Dim ret As Boolean  
  
    Set swApp = Application.SldWorks  
    Set swModelDoc = swApp.ActiveDoc  
    Set swAssemblyDoc = swModelDoc  
  
    'Open the Interference Detection pane  
    swAssemblyDoc.ToolsCheckInterference  
  
    Set pIntMgr = swAssemblyDoc.InterferenceDetectionManager  
  
    'Specify the interference detection settings and options  
    pIntMgr.TreatCoincidenceAsInterference = False  
    pIntMgr.TreatSubAssembliesAsComponents = False  
    pIntMgr.IncludeMultibodyPartInterferences = True  
    pIntMgr.MakeInterferingPartsTransparent = True  
    pIntMgr.CreateFastenersFolder = False  
    pIntMgr.IgnoreHiddenBodies = False  
    pIntMgr.ShowIgnoredInterferences = False  
    pIntMgr.UseTransform = False  
  
    'Specify how to display non-interfering components  
    pIntMgr.NonInterferingComponentDisplay = swNonInterferingComponentDisplay_Wireframe
```

```

'Run interference detection
vInts = pIntMgr.GetInterferences
Debug.Print "Number of interferences: " & pIntMgr.GetInterferenceCount

'Get interfering components and transforms
ret = pIntMgr.GetComponentsAndTransforms(vComps, vTrans)

'Get interference information
For i = 0 To UBound(vInts)
    Debug.Print "Interference " & (i + 1)
    Set interference = vInts(i)
    Debug.Print "    Number of components in this interference: " &
interference.GetComponentCount
    vComps = interference.Components
    For j = 0 To UBound(vComps)
        Set comp = vComps(j)
        Debug.Print "        " & comp.Name2
    Next j
    vol = interference.Volume
    Debug.Print "    Interference volume is " & (vol * 1000000000) & " mm^3"
Next i

ret = pIntMgr.ExportResults("c:\temp\IntDetReport.xlsx", True)

'Stop interference detection and close Interference Detection pane
'pIntMgr.Done

End Sub

```

Run Interference Detection Example (VB.NET)

This example shows how to run interference detection and observe the interferences in an assembly.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\advdrawings\ Bowl and chute.sldasm.  
' 2. Ensure c:\temp exists.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Opens the Interference Detection pane.  
' 2. Gets interferences.  
' 3. Gets interfering components and transforms.  
' 4. Gets interference information.  
' 5. In the Results list on the Interference Detection pane,  
'    click on each interference and observe it in the graphics area (red).  
' 6. To close the Interference Detection pane, right-click the graphics area  
'    and select Cancel.  
' 7. Inspect the Immediate window and c:\temp\IntDetReport.xlsx.  
'  
' NOTE: Because the assembly is used elsewhere, do not save changes.  
'-----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System  
Imports System.Diagnostics  
Partial Class SolidWorksMacro  
  
    Sub main()  
  
        Dim swModelDoc As ModelDoc2  
        Dim swAssemblyDoc As AssemblyDoc  
        Dim pIntMgr As InterferenceDetectionMgr  
        Dim vInts As Object  
        Dim i As Long  
        Dim j As Long  
        Dim interference As IInterference  
        Dim vComps As Object = Nothing  
        Dim comp As Component2  
        Dim vol As Double  
        Dim vTrans As Object = Nothing  
        Dim ret As Boolean  
  
        swModelDoc = swApp.ActiveDoc  
        swAssemblyDoc = swModelDoc  
  
        'Open the Interference Detection pane  
        swAssemblyDoc.ToolsCheckInterference  
  
        pIntMgr = swAssemblyDoc.InterferenceDetectionManager  
  
        'Specify the interference detection settings and options  
        pIntMgr.TreatCoincidenceAsInterference = False  
        pIntMgr.TreatSubAssembliesAsComponents = False  
        pIntMgr.IncludeMultibodyPartInterferences = True  
        pIntMgr.MakeInterferingPartsTransparent = True  
        pIntMgr.CreateFastenersFolder = False  
        pIntMgr.IgnoreHiddenBodies = False  
        pIntMgr.ShowIgnoredInterferences = False  
    End Sub  
End Class
```



```

pIntMgr.UseTransform = False

'Specify how to display non-interfering components
pIntMgr.NonInterferingComponentDisplay =
swNonInterferingComponentDisplay_e.swNonInterferingComponentDisplay_Wireframe

'Run interference detection
vInts = pIntMgr.GetInterferences
Debug.Print("# of interferences: " & pIntMgr.GetInterferenceCount)

'Get interfering components and transforms
ret = pIntMgr.GetComponentsAndTransforms(vComps, vTrans)

'Get interference information
For i = 0 To UBound(vInts)

    Debug.Print("Interference " & (i + 1))
    interference = vInts(i)
    Debug.Print("  Number of components in this interference: " &
interference.GetComponentCount)
    vComps = interference.Components
    For j = 0 To UBound(vComps)
        comp = vComps(j)
        Debug.Print("    " & comp.Name2)
    Next j
    vol = interference.Volume
    Debug.Print("  Interference volume is " & (vol * 1000000000) & "
mm^3")
Next i

ret = pIntMgr.ExportResults("c:\temp\IntDetReport.xlsx", True)

'Stop interference detection and close Interference Detection pane
pIntMgr.Done()
End Sub
Public swApp As SldWorks

End
Class

```

Run Macro

This example shows how to use ISldWorks::RunMacro.

You can only:

- Execute a Sub that takes no arguments and that is not part of a class or form.
- Open forms from within a Sub; you cannot start a form directly.

```
'-----  
'  
' RunMacroMain.swp  
'  
'-----  
  
Option Explicit  
  
  
Dim swApp As SldWorks.SldWorks  
Dim boolstatus As Boolean  
  
Sub main()  
  
Set swApp = Application.SldWorks  
  
Dim filename As String  
filename = swApp.GetCurrentMacroPathName  
filename = Left(filename, InStrRev(filename, "\")) + "RunMacroSub.swp"  
boolstatus = swApp.RunMacro(filename, "RunMacroSub1", "alternate")  
boolstatus = swApp.RunMacro(filename, "RunMacroSub1", "main")  
End Sub
```

```
'-----  
'  
' RunMacroSub.swp  
'  
'-----  
  
Option Explicit  
  
  
Dim swApp As SldWorks.SldWorks  
  
Sub alternate()  
  
Set swApp = Application.SldWorks  
swApp.SendMsgToUser "RunMacroSub1:alternate() called"  
End Sub  
  
Sub main()  
  
Set swApp = Application.SldWorks  
swApp.SendMsgToUser "RunMacroSub1:main() called"
```

End Sub

Run SOLIDWORKS Commands and Synthesize Mouse Events Example (C#)

This example shows how to run SOLIDWORKS commands and synthesize mouse events.

```
//-----
// Preconditions:
// 1. Verify that the specified part document to open exists.
// 2. Click Tools > Options and click to clear the Stop VSTA debugger
//    on macro exit check box, if it is selected.
// 3. In the IDE, right-click the project name in the Project Explorer,
//    click Add Reference, browse install_dir\api\redist, click
//    SolidWorks.Interop.swcommands.dll, and click OK.
// 4. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified part document.
// 2. Fits the model to the graphics area.
// 3. Moves the mouse.
// 4. Changes the view orientation to *Front.
// 5. Select an edge on the model.
// 6. Left-click anywhere outside the model in the graphics area.
// 7. Examine the Immediate window.
// 8. Click Stop Debugging in the IDE.
// 9. Click Tools > Options and click to select the Stop VSTA debugger
//    on macro exit check box, if it is clear.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using SolidWorks.Interop.swcommands;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace MouseCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public Mouse msMouse;

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            ModelView swModelView = default(ModelView);
            Mouse TheMouse = default(Mouse);
            int i = 0;
            double x = 0;
            string fileName = null;
            int errors = 0;
            int warnings = 0;
            bool status = false;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\api\\block20.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e
            .swOpenDocOptions_Silent, "", ref errors, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swModelView = (ModelView)swModel.GetFirstModelView();
            TheMouse = (Mouse)swModelView.GetMouse();

            //Set up events
            msMouse = (Mouse)TheMouse;
            AttachEventHandlers();

            x = 0;

            Debug.Print("Fit model to graphics area");
            swModelDocExt.RunCommand((int)swCommands_e.swCommands_ZoomToFit, "");

            //Move the mouse
            status = TheMouse.Move(50, 150, (int)swMouse_e.swMouse_Absolute + (int)swMouse_e.swMouse_MouseMove + (i
            nt)swMouse_e.swMouse_LeftDown);
            Debug.Print("First call to Move: " + status);
            Debug.Print("Calls to Move within loop:");
            for (i = 1; i <= 5; i++)
            {
                status = TheMouse.Move(5, 5, (int)swMouse_e.swMouse_MouseMove);
                Debug.Print(" Call " + i + " to Move: " + status);
            }
            status = TheMouse.Move(0, 0, (int)swMouse_e.swMouse_LeftUp);
        }
    }
}
```

```

        Debug.Print("Last call to Move: " + status);
        status = TheMouse.MoveXYZ(0.03720615681732, 0.0316583060694, 0.04991700841805, (int)swMouse_e.swMouse_LeftDown);
        Debug.Print("Call to MoveXYZ: " + status);
        Debug.Print("Calls to Move within next loop:");
        for (i = 1; i <= 5; i++)
        {
            status = TheMouse.Move(5, 5, (int)swMouse_e.swMouse_MouseMove);
            Debug.Print("  Call " + (i + 1).ToString() + " to Move: " + status);
        }

        status = TheMouse.Move(10, 10, (int)swMouse_e.swMouse_LeftUp);
        Debug.Print("Last call to Move: " + status);

        Debug.Print("Change view to *Front");
        swModelDocExt.RunCommand((int)swCommands_e.swCommands_Front, "");
    }

    public void AttachEventHandlers()
    {
        AttachSWEvents();
    }

    public void AttachSWEvents()
    {
        msMouse.MouseSelectNotify += this.ms_MouseSelectNotify;
        msMouse.MouseLBtnDownNotify += this.ms_MouseLBtnDownNotify;
    }

    private int ms_MouseSelectNotify(int Ix, int Iy, double x, double y, double z)
    {
        Debug.Print("Selection made:");
        Debug.Print("  Mouse location:");
        Debug.Print("    Window space coordinates:");
        Debug.Print("      " + Ix);
        Debug.Print("      " + Iy);
        Debug.Print("    World space coordinates:");
        Debug.Print("      " + x);
        Debug.Print("      " + y);
        Debug.Print("      " + z);

        return 1;
    }

    private int ms_MouseLBtnDownNotify(int x, int y, int WParam)
    {
        Debug.Print("Left-mouse button pressed.");

        return 1;
    }

    /// <summary>
    ///   The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}

```

Run SOLIDWORKS Commands and Synthesize Mouse Events Example (VBA)

This example shows how to run SOLIDWORKS commands and synthesize mouse events.

```
'-----  
' Preconditions:  
' 1. Verify that the specified part document to open exists.  
' 2. Right-click the project name in the Project Explorer and click  
'    Insert > Class Module.  
' 3. Copy Module to the main module.  
' 4. Copy Class1 to the class module.  
' 5. In the IDE, click Tools > References, click  
'    SOLIDWORKS version Commands type library, and click OK.  
' 6. Open the Immediate window.  
'  
' Postconditions:  
' 1. Opens the specified part document.  
' 2. Fits the model to the graphics area.  
' 3. Moves the mouse.  
' 4. Changes the view orientation to *Front.  
' 5. Select an edge on the model.  
' 6. Left-click anywhere outside the model in the graphics area.  
' 7. Click Reset in the IDE to stop executing the macro.  
' 8. Examine the Immediate window.  
'-----  
'Module  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swModelView As SldWorks.ModelView  
Dim swMouse As SldWorks.mouse  
Dim TheMouse As SldWorks.mouse  
Dim obj As New Class1  
Dim i As Long  
Dim x As Double  
Dim fileName As String  
Dim errors As Long  
Dim warnings As Long  
Dim status As Boolean  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\block20.sldprt"  
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART,  
swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)  
    Set swModelDocExt = swModel.Extension  
    Set swModelView = swModel.GetFirstModelView  
    Set TheMouse = swModelView.GetMouse  
  
    obj.init TheMouse  
  
    x = 0  
  
    Debug.Print "Fit model to graphics area"  
    swModelDocExt.RunCommand swCommands_ZoomToFit, ""  
  
    'Move the mouse  
    status = TheMouse.Move(50, 150, swMouse_Absolute + swMouse_MouseMove + swMouse_LeftDown)  
    Debug.Print ("First call to Move: " & status)  
  
    Debug.Print "Calls to Move within loop:"  
    For i = 1 To 5  
        DoEvents  
        status = TheMouse.Move(5, 5, swMouse_MouseMove)  
        Debug.Print ("  Call " & i & " to Move: " & status)  
    Next
```

```

status = TheMouse.Move(0, 0, swMouse_LeftUp)
Debug.Print ("Last call to Move: " & status)

Debug.Print("Call to MoveXYZ: " & status)
status = TheMouse.MoveXYZ(0.03720615681732, 0.0316583060694, 0.04991700841805, swMouse_LeftDown)
Debug.Print ("Calls to Move within next loop: ")

For i = 1 To 5
    DoEvents
    status = TheMouse.Move(5, 5, swMouse_MouseMove)
    Debug.Print "    Call " & i + 1; " to Move: " & " & status
Next

    status = TheMouse.Move(0, 0, swMouse_LeftUp)
    Debug.Print ("Call to Move after loop: " & status)

Debug.Print "Change view orientation to *Front"
swModelDocExt.RunCommand swCommands_Front, ""

End Sub

```

[Back to top](#)

```
'Class1
```

```
Dim WithEvents ms As SldWorks.mouse
```

```
Private Sub Class_Initialize()
End Sub
```

```
Public Sub init(mouse As Object)
    Set ms = mouse
End Sub
```

```
Private Function ms_MouseSelectNotify(ByVal lx As Long, ByVal ly As Long, ByVal x As Double, ByVal y As Double, ByVal z As Double) As Long
    Debug.Print "Selection made:"
    Debug.Print " Mouse location:"
    Debug.Print "   Window space coordinates:"
    Debug.Print "     " & lx
    Debug.Print "     " & ly
    Debug.Print "   World space coordinates:"
    Debug.Print "     " & x
    Debug.Print "     " & y
    Debug.Print "     " & z
End Function
```

```
Private Function ms_MouseLBtnDownNotify(ByVal x As Long, ByVal y As Long, ByVal WParam As Long) As Long
    Debug.Print "Left-mouse button pressed."
End Function
```

[Back to top](#)

Run SOLIDWORKS Commands and Synthesize Mouse Events Example (VB.NET)

This example shows how to run SOLIDWORKS commands and synthesize mouse events.

```

'-----
' Preconditions:
' 1. Verify that the specified part document to open exists.
' 2. Click Tools > Options and click to clear the Stop VSTA debugger
'    on macro exit check box, if it is selected.
' 3. In the IDE, right-click the project name in the Project Explorer,
'    click Add Reference, browse install_dir\api\redist, click
'    SolidWorks.Interop.swcommands.dll, and click OK.
' 4. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified part document.
' 2. Fits the model to the graphics area.
' 3. Moves the mouse.
' 4. Changes the view orientation to *Front.
' 5. Select an edge on the model.
' 6. Left-click anywhere outside the model in the graphics area.
' 7. Examine the Immediate window.
' 8. Click Stop Debugging in the IDE.
' 9. Click Tools > Options and click to select the Stop VSTA debugger
'    on macro exit check box, if it is clear.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports SolidWorks.Interop.swcommands
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

```

Partial Class SolidWorksMacro

Public WithEvents msMouse As Mouse

Public Sub main()

```

    Dim swModel As ModelDoc2
    Dim swModelDocExt As ModelDocExtension
    Dim swModelView As ModelView
    Dim TheMouse As Mouse
    Dim i As Integer
    Dim x As Double
    Dim fileName As String
    Dim errors As Integer
    Dim warnings As Integer
    Dim status As Boolean

```

```

    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\block20.sldprt"
    swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent,
    "", errors, warnings)
    swModelDocExt = swModel.Extension
    swModelView = swModel.GetFirstModelView
    TheMouse = swModelView.GetMouse

```

```

    'Set up events
    msMouse = TheMouse
    AttachEventHandlers()

```

x = 0

```

    Debug.Print("Fit model to graphics area")
    swModelDocExt.RunCommand(swCommands_e.swCommands_ZoomToFit, "")

```

'Move the mouse

```

    status = TheMouse.Move(50, 150, swMouse_e.swMouse_Absolute + swMouse_e.swMouse_MouseMove + swMouse_e.swMouse_
e_LeftDown)

```

```

    Debug.Print("First call to Move: " & status)

```

```

    Debug.Print("Calls to Move within loop:")

```

```

    For i = 1 To 5

```

```

        status = TheMouse.Move(5, 5, swMouse_e.swMouse_MouseMove)

```

```

        Debug.Print(" Call " & i & " to Move: " & status)

```

```

    Next

```

```

    status = TheMouse.Move(0, 0, swMouse_e.swMouse_LeftUp)

```

```

    Debug.Print("Last call to Move: " & status)

```

```

    status = TheMouse.MoveXYZ(0.03720615681732, 0.0316583060694, 0.04991700841805, swMouse_e.swMouse_LeftDown)

```

```

    Debug.Print("Call to MoveXYZ: " & status)

```



```

Debug.Print("Calls to Move within next loop:")
For i = 1 To 5
    status = TheMouse.Move(5, 5, swMouse_e.swMouse_MouseMove)
    Debug.Print(" Call " & (i + 1).ToString & " to Move: " & status)
Next

status = TheMouse.Move(10, 10, swMouse_e.swMouse_LeftUp)
Debug.Print("Last call to Move: " & status)

Debug.Print("Change view to *Front")
swModelDocExt.RunCommand(swCommands_e.swCommands_Front, "")

End Sub

Sub AttachEventHandlers()
    AttachSwEvents()
End Sub

Sub AttachSwEvents()

    AddHandler msMouse.MouseSelectNotify, AddressOf Me.ms_MouseSelectNotify
    AddHandler msMouse.MouseLBtnDownNotify, AddressOf Me.ms_MouseLBtnDownNotify

End Sub

Private Function ms_MouseSelectNotify(ByVal Ix As Integer, ByVal Iy As Integer, ByVal x As Double, ByVal y As Double, ByVal z As Double) As Integer
    Debug.Print("Selection made:")
    Debug.Print(" Mouse location:")
    Debug.Print(" Window space coordinates:")
    Debug.Print(" " & Ix)
    Debug.Print(" " & Iy)
    Debug.Print(" World space coordinates:")
    Debug.Print(" " & x)
    Debug.Print(" " & y)
    Debug.Print(" " & z)
End Function

Private Function ms_MouseLBtnDownNotify(ByVal x As Integer, ByVal y As Integer, ByVal WParam As Integer) As Integer
    Debug.Print("Left-mouse button pressed.")
End Function

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Run SOLIDWORKS Commands Example (VBA)

This example shows how to programmatically open a Fillet PropertyManager page titled **My Fillet**. The macro also programmatically changes the view orientation of the part to ***Front**.

```
'-----  
' Preconditions:  
' 1. In IDE, click Tools > References, and click SOLIDWORKS  
'    version Commands type library, and click OK.  
' 2. Open a part in SOLIDWORKS Design.  
' 3. Change the view orientation of the part to any view  
'    except *Front.  
' 4. Select the edges on the part to fillet.  
'  
' Postconditions:  
' 1. Opens My Fillet PropertyManager page.  
' 2. Click the OK button on the My Fillet PropertyManager to  
'    fillet the selected edges.  
' 3. Click Continue (F5) in the IDE.  
' 4. Changes the view orientation *Front.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension
```

```
Sub main()
```

```
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swModelDocExt = swModel.Extension
```

```
    'Open Fillet PropertyManager page  
    'Title of page is My Fillet  
    swModelDocExt.RunCommand swCommands_Fillet, "My Fillet"
```

```
    Stop  
    'Click OK in the My Fillet PropertyManager page to  
    'create the fillets  
    'Click Continue (F5) in the IDE
```

```
    'Change the view orientation of the part to *Front  
    swModelDocExt.RunCommand swCommands_Front, ""
```

```
End Sub
```

Save as De-Featured File Example (C#)

This example shows how to de-feature an assembly and save it as a part.

```
//-----  
// Preconditions:  
// 1. Open public_documents\samples\tutorial\flexpress\ball_valve\ball_valve.sldasm.  
// 2. Verify that c:\temp exists.  
//  
// Postconditions:  
// 1. Saves the assembly as a de-featured part.  
// 2. Open c:\temp\ball_valve.sldprt to verify.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System;  
  
namespace SaveFileAsDeFeaturedPart_CSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
        ModelDoc2 swModel;  
        bool boolstatus;  
  
        public void Main()  
        {  
            swModel= (ModelDoc2)swApp.ActiveDoc;  
            boolstatus = swModel.Extension.SaveDeFeaturedFile("C:\\temp\\ball_valve.sldprt");  
        }  
  
        public SldWorks swApp;  
    }  
}
```

Save_As_Defeatured_File_Example_VB (Topic)

SOLIDWORKS API Help

Save as De-Featured File Example (VBA)

This example shows how to de-feature an assembly and save it as a part.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\floypress\ball valve\ball_valve.sldasm.  
' 2. Verify that c:\temp exists.  
'  
' Postconditions:  
' 1. Saves the assembly as a de-featured part.  
' 2. Open c:\temp\ball_valve.sldprt to verify.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim boolstatus As Boolean
```

```
Sub main()
```

```
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    boolstatus = swModel.Extension.SaveDeFeaturedFile("c:\temp\ball_valve.sldprt")
```

```
End Sub
```

Save as De-Featured File Example (VB.NET)

This example shows how to de-feature an assembly and save it as a part.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\floypress\ball_valve\ball_valve.sldasm.  
' 2. Verify that c:\temp exists.  
'  
' Postconditions:  
' 1. Saves the assembly as a de-featured part.  
' 2. Open c:\temp\ball_valve.sldprt to verify.  
'-----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Dim swModel As ModelDoc2  
    Dim boolstatus As Boolean
```

```
    Sub main()
```

```
        swModel = swApp.ActiveDoc  
        boolstatus = swModel.Extension.SaveDeFeaturedFile("c:\temp\ball_valve.sldprt")
```

```
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

Save Document As TIFF Example (VBA)

This example shows how to:

- Set TIFF export options
- Save the active document to a .tiff format in the same directory as the model document.

```
'-----  
' Preconditions:  
' 1. Open a model document.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Gets TIFF settings.  
' 2. Saves the model document as a TIFF file to the same folder as  
'    the model document and overwrites any existing file of the same name  
'    in that folder.  
' 3. Examine the Immediate window and the folder where the TIFF file  
'    was saved.  
'-----  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim sPathName As String  
    Dim nRetVal As Long  
    Dim nErrors As Long  
    Dim nWarnings As Long  
    Dim bRe As Boolean  
  
    Set swApp = CreateObject("SldWorks.Application")  
    Set swModel = swApp.ActiveDoc  
  
    ' Get current TIFF settings  
    Debug.Print "PrintScaleToFit" = " + Str(swApp.GetUserPreferenceToggle(swTiffPrintScaleToFit))  
    Debug.Print "ScreenOrPrintCapture" = " + Str(swApp.GetUserPreferenceIntegerValue(swTiffScreenOrPrintCapture))  
    Debug.Print "ImageType" = " + Str(swApp.GetUserPreferenceIntegerValue(swTiffImageType))  
    Debug.Print "CompressionScheme" = " + Str(swApp.GetUserPreferenceIntegerValue(swTiffCompressionScheme))  
    Debug.Print "PrintDPI" = " + Str(swApp.GetUserPreferenceIntegerValue(swTiffPrintDPI))  
    Debug.Print "PrintPaperSize" = " + Str(swApp.GetUserPreferenceIntegerValue(swTiffPrintPaperSize))  
    Debug.Print "PrintScaleFactor" = " + Str(swApp.GetUserPreferenceIntegerValue(swTiffPrintScaleFactor))  
    Debug.Print "DrawingPaperHeight" = " +  
Str(swApp.GetUserPreferenceDoubleValue(swTiffPrintDrawingPaperHeight))  
    Debug.Print "DrawingPaperWidth" = " +  
Str(swApp.GetUserPreferenceDoubleValue(swTiffPrintDrawingPaperWidth))  
  
    ' Strip off SOLIDWORKS Design file name extension (sldprt, sldasm, or slddrw)  
    ' and add TIFF extension (tif)  
    sPathName = swModel.GetPathName  
    sPathName = Left(sPathName, Len(sPathName) - 6)  
    sPathName = sPathName + ".tif"  
    Debug.Print sPathName  
  
    bRet = swModel.SaveAs4(sPathName, swSaveAsCurrentVersion, swSaveAsOptions_Silent, nErrors, nWarnings)  
    If bRet = False Then  
        nRetVal = swApp.SendMsgToUser2("Problems saving file.", swMbWarning, swMbOk)  
    End If  
  
End Sub
```

Save_Configuration_Data_Example_CSharp (Topic)

SOLIDWORKS API Help

Save Configuration Data Example (C#)

This example shows how to mark each configuration in a multi-configuration model as needing to be rebuilt and how to save its configuration data, including preview bitmaps, every time you save the model document.

```
//-----  
// Preconditions:  
// 1. Verify that the specified file to open exists.  
// 2. Open the Immediate window.  
//  
// Postconditions:  
// 1. Activates each configuration in the model.  
// 2. Sets each configuration's Rebuild/Save Mark to true, if it is false.  
// 3. Saves each configuration's preview bitmap to disk.  
// 4. Examine the Immediate window.  
//  
// NOTES:  
// * In SOLIDWORKS 2013 and later, to mark each configuration  
// as needing to be rebuilt and to save its configuration data  
// every time you save the model:  
// 1. Activate each configuration and set  
// IConfiguration::AddRebuildSaveMark to true.  
// 2. Save the model.  
// * Because this model is used elsewhere, do not  
// save changes.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
using System;  
using System.Diagnostics;  
  
namespace AddRebuildSaveMarkCSharp.csproj  
{  
    public partial class SolidWorksMacro  
    {  
  
        public void Main()  
        {  
  
            ModelDoc2 swModel = default(ModelDoc2);  
            Configuration swConfig = default(Configuration);  
            ConfigurationManager swConfMgr = default(ConfigurationManager);  
            object[] configNameArr = null;  
            string configName = null;  
            string fileName = null;  
            string bitmapName = null;  
            string bitmapPathName = null;  
            bool status = false;  
            int errors = 0;  
            int warnings = 0;  
            int i = 0;  
  
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS  
2018\\samples\\tutorial\\api\\multiconfig-part-2.sldprt";  
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e  
.swOpenDocOptions_Silent, "", ref errors, ref warnings);  
            swConfMgr = (ConfigurationManager)swModel.ConfigurationManager;  
            configNameArr = (object[])swModel.GetConfigurationNames();  
  
            Debug.Print("Checking each configuration's Add Rebuild/Save Mark setting after opening the model docume  
nt:");  
  
            for (i = 0; i <= configNameArr.GetUpperBound(0); i++)  
            {  
                configName = (string)configNameArr[i];  
                swConfig = (Configuration)swModel.GetConfigurationByName(configName);  
                status = swModel.ShowConfiguration2(configName);  
                Debug.Print(" " + configName + "'s" + " Add Rebuild/Save Mark = " + swConfig.AddRebuildSaveMark);  
                if (swConfig.AddRebuildSaveMark == false)  
                {  
                    swConfig.AddRebuildSaveMark = true;  
                    bitmapName = configName + ".bmp";  
                    Debug.Print(" Resetting " + configName + "'s" + " Add Rebuild/Save Mark = " + swConfig.Add
```

```

RebuildSaveMark);
    bitmapPathName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\api\\" + bitmapName;
    status = swApp.GetPreviewBitmapFile(fileName, configName, bitmapPathName);
    if (status)
    {
        Debug.Print(" " + configName + "'s " +
"preview bitmap written to disk: " + bitmapPathName);
    }
}

//Save the model to save each configuration's data with the model
//status = swModel.Save3(swSaveAsOptions_e.swSaveAsOptions_Silent, errors, warnings)

}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```


Save Configuration Data Example (VBA)

This example shows how to mark each configuration in a multi-configuration model as needing to be rebuilt and how to save its configuration data, including preview bitmaps, every time you save the model document.

```
'-----  
' Preconditions:  
' 1. Verify that the specified file to open exists.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Activates each configuration in the model.  
' 2. Sets each configuration's Rebuild/Save Mark to true, if it is false.  
' 3. Saves each configuration's preview bitmap to disk.  
' 4. Examine the Immediate window.  
'  
' NOTES:  
' * In SOLIDWORKS 2013 and later, to mark each configuration  
'   as needing to be rebuilt and to save its configuration data  
'   every time you save the model:  
'   1. Activate each configuration and set  
'      IConfiguration::AddRebuildSaveMark to true.  
'   2. Save the model.  
' * Because this model is used elsewhere, do not  
'   save changes.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swConfig As SldWorks.Configuration  
Dim swConfMgr As SldWorks.ConfigurationManager  
Dim configNameArr As Variant  
Dim configName As Variant  
Dim fileName As String  
Dim bitmapName As String  
Dim bitmapPathName As String  
Dim status As Boolean  
Dim errors As Long, warnings As Long
```

Sub main()

```
    Set swApp = Application.SldWorks  
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\multiconfig-part-  
2.sldprt"  
    Set swModel = swApp.OpenDoc6(fileName, swDocPART, swOpenDocOptions_Silent, "", errors, warnings)  
    Set swConfMgr = swModel.ConfigurationManager  
    configNameArr = swModel.GetConfigurationNames  
  
    Debug.Print "Checking each configuration's Add Rebuild/Save Mark setting after opening the model  
document:"  
  
    For Each configName In configNameArr  
        Set swConfig = swModel.GetConfigurationByName(configName)  
        status = swModel.ShowConfiguration2(configName)  
        Debug.Print (" " & configName & "'s" & " Add Rebuild/Save Mark = " & swConfig.AddRebuildSaveMark)  
        If swConfig.AddRebuildSaveMark = False Then  
            swConfig.AddRebuildSaveMark = True  
            bitmapName = configName & ".bmp"  
            Debug.Print "      Resetting " & configName & "'s" & " Add Rebuild/Save Mark = " &  
swConfig.AddRebuildSaveMark  
            bitmapPathName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\" +  
bitmapName  
            status = swApp.GetPreviewBitmapFile(fileName, configName, bitmapPathName)  
            If status Then  
                Debug.Print " " & configName & "'s " & "preview bitmap written to disk: " & bitmapPathName  
            End If  
        End If  
    Next
```

```
'Save the model to save each configuration's data with the model  
'status = swModel.Save3(swSaveAsOptions_e.swSaveAsOptions_Silent, errors, warnings)
```

```
End Sub
```

Save Configuration Data Example (VB.NET)

This example shows how to mark each configuration in a multi-configuration model as needing to be rebuilt and how to save its configuration data, including preview bitmaps, every time you save the model document.

```

'-----
' Preconditions:
' 1. Verify that the specified file to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Activates each configuration in the model.
' 2. Sets each configuration's Rebuild/Save Mark to true, if it is false.
' 3. Saves each configuration's preview bitmap to disk.
' 4. Examine the Immediate window.
'
' NOTES:
' * In SOLIDWORKS 2013 and later, to mark each configuration
'   as needing to be rebuilt and to save its configuration data
'   every time you save the model:
'   1. Activate each configuration and set
'      IConfiguration::AddRebuildSaveMark to true.
'   2. Save the model.
' * Because this model is used elsewhere, do not
'   save changes.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swConfig As Configuration
        Dim swConfMgr As ConfigurationManager
        Dim configNameArr As Object
        Dim configName As Object
        Dim fileName As String
        Dim bitmapName As String
        Dim bitmapPathName As String
        Dim status As Boolean
        Dim errors As Integer, warnings As Integer

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\multiconfig-part-
2.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent,
"", errors, warnings)
        swConfMgr = swModel.ConfigurationManager
        configNameArr = swModel.GetConfigurationNames

        Debug.Print("Checking each configuration's Add Rebuild/Save Mark setting after opening the model document:")

    )

    For Each configName In configNameArr
        swConfig = swModel.GetConfigurationByName(configName)
        status = swModel.ShowConfiguration2(configName)
        Debug.Print(" " & configName & "'s" & " Add Rebuild/Save Mark = " & swConfig.AddRebuildSaveMark)
        If swConfig.AddRebuildSaveMark = False Then
            swConfig.AddRebuildSaveMark = True
            bitmapName = configName & ".bmp"
            Debug.Print("      Resetting " & configName & "'s" & " Add Rebuild/Save Mark = " & swConfig.AddRebu
ildSaveMark)
            bitmapPathName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\api\" + bitmapName
            status = swApp.GetPreviewBitmapFile(fileName, configName, bitmapPathName)
            If status Then
                Debug.Print("      " & configName & "'s" &
"preview bitmap written to disk: " & bitmapPathName)
            End If
        End If
    Next

    'Save the model to save each configuration's data with the model
    'status = swModel.Save3(swSaveAsOptions_e.swSaveAsOptions_Silent, errors, warnings)

```

End Sub

```
''' <summary>  
''' The SldWorks swApp variable is pre-assigned for you.  
''' </summary>  
Public swApp As SldWorks
```

End Class

Save Design Table as Microsoft Excel File Example (VBA)

This example shows how to save a design table to a Microsoft Excel file.

```
'-----  
'  
' Preconditions: Document that has a design table is open  
'  
' Postcoditions: Design table is saved as a Microsoft  
'           Excel file named DesignTable.xls  
'  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
  
Sub main()  
  
    Dim swModel      As SldWorks.ModelDoc2  
    Dim swDesignTable As SldWorks.DesignTable  
    Dim strFileName   As String  
    Dim bValue        As Boolean  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
  
    Set swDesignTable = swModel.GetDesignTable  
  
    swDesignTable.Attach  
  
    bValue = swDesignTable.SaveAsExcelFile(swApp.GetCurrentMacroPathFolder & "\DesignTable.xls")  
  
    ' Do not detach if table is not active  
    If (Not (swDesignTable.IsActive = False)) Then  
        swDesignTable.Detach  
    End If  
  
End Sub
```

Save Drawing as DXF Example (C#)

This example shows how to save the current drawing file as a DXF file in the same folder.

```
//-----  
// Preconditions:  
// 1. Open a drawing.  
// 2. Open the Immediate window.  
//  
// Postconditions:  
// 1. Gets and sets DXF-related system settings.  
// 2. Saves the drawing as a DXF file in the same folder as the drawing,  
//    overwriting any existing file of the same name.  
// 3. Examine the Immediate window and the folder to which the DXF file  
//    was saved.  
//-----  
  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
using System;  
using System.Diagnostics;  
namespace SaveDrawingAsDXF_CSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
  
        ModelDoc2 swModel;  
        string sPathName;  
        int nErrors;  
        int nWarnings;  
        int nRetVal;  
        bool bShowMap;  
  
        bool bRet;  
  
        public void Main()  
        {  
            swModel = (ModelDoc2)swApp.ActiveDoc;  
  
            // Strip off SOLIDWORKS Design drawing file extension (.slddrw)  
            // and add DXF file extension (.dxf)  
            sPathName = swModel.GetPathName();  
            sPathName = sPathName.Substring(0, sPathName.Length - 6);  
            sPathName = sPathName + "dxf";  
            Debug.Print(sPathName);  
  
            // Show current settings  
            Debug.Print("DxfMapping          = " +  
swApp.GetUserPreferenceToggle((int)swUserPreferenceToggle_e.swDxfMapping));  
            Debug.Print("DXFDontShowMap      = " +  
swApp.GetUserPreferenceToggle((int)swUserPreferenceToggle_e.swDXFDontShowMap));  
            Debug.Print("DxfVersion          = " +  
swApp.GetUserPreferenceIntegerValue((int)swUserPreferenceIntegerValue_e.swDxfVersion));  
            Debug.Print("DxfOutputFonts       = " +  
swApp.GetUserPreferenceIntegerValue((int)swUserPreferenceIntegerValue_e.swDxfOutputFonts));  
            Debug.Print("DxfMappingFileIndex  = " +  
swApp.GetUserPreferenceIntegerValue((int)swUserPreferenceIntegerValue_e.swDxfMappingFileIndex));  
            Debug.Print("DxfOutputLineStyles  = " +  
swApp.GetUserPreferenceIntegerValue((int)swUserPreferenceIntegerValue_e.swDxfOutputLineStyles));  
            Debug.Print("DxfOutputNoScale     = " +  
swApp.GetUserPreferenceIntegerValue((int)swUserPreferenceIntegerValue_e.swDxfOutputNoScale));  
            Debug.Print("DxfMappingFiles      = " +  
swApp.GetUserPreferenceStringListValue((int)swUserPreferenceStringListValue_e.swDxfMappingFiles));  
            Debug.Print("DxfOutputScaleFactor = " +
```

```

swApp.GetUserPreferenceDoubleValue((int)swUserPreferenceDoubleValue_e.swDxfOutputScaleFactor));
    Debug.Print("");

    // Turn off showing of map
    bShowMap =
swApp.GetUserPreferenceToggle((int)swUserPreferenceToggle_e.swDXFDontShowMap);
    Debug.Print("bShowMap = " + bShowMap);

    swApp.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swDXFDontShowMap, false);

    bRet = swModel.SaveAs4(sPathName, (int)swSaveAsVersion_e.swSaveAsCurrentVersion,
(int)swSaveAsOptions_e.swSaveAsOptions_Silent, ref nErrors, ref nWarnings);

    if (bRet == false)
    {
        nRetval = swApp.SendMsgToUser2("Problems saving file.",
(int)swMessageBoxIcon_e.swMbWarning, (int)swMessageBoxBtn_e.swMbOk);
    }

    // Restore showing of map
    swApp.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swDXFDontShowMap,
bShowMap);
}

    public SldWorks swApp;
}
}

```

Save Drawing as DXF Example (VBA)

This example shows how to save the current drawing file as a DXF file in the same folder.

```
'-----  
' Preconditions:  
' 1. Open a drawing.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Gets and sets DXF-related system settings.  
' 2. Saves the drawing as a DXF file in the same folder as the drawing,  
'    overwriting any existing file of the same name.  
' 3. Examine the Immediate window and the folder to which the DXF file  
'    was saved.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim sPathName As String  
Dim nErrors As Long  
Dim nWarnings As Long  
Dim nRetVal As Long  
Dim bShowMap As Boolean  
Dim bRet As Boolean  
  
Sub main()  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
  
    ' Strip off SOLIDWORKS Design drawing file extension (.slddrw)  
    ' and add DXF file extension (.dxf)  
    sPathName = swModel.GetPathName  
    sPathName = Left(sPathName, Len(sPathName) - 6)  
    sPathName = sPathName + ".dxf"  
  
    ' Show current settings  
    Debug.Print "DxfMapping" = " & swApp.GetUserPreferenceToggle(swDxfMapping)  
    Debug.Print "DXFDontShowMap" = " & swApp.GetUserPreferenceToggle(swDXFDontShowMap)  
    Debug.Print "DxfVersion" = " & swApp.GetUserPreferenceIntegerValue(swDxfVersion)  
    Debug.Print "DxfOutputFonts" = " &  
    swApp.GetUserPreferenceIntegerValue(swDxfOutputFonts)  
    Debug.Print "DxfMappingFileIndex" = " &  
    swApp.GetUserPreferenceIntegerValue(swDxfMappingFileIndex)  
    Debug.Print "DxfOutputLineStyles" = " &  
    swApp.GetUserPreferenceIntegerValue(swDxfOutputLineStyles)  
    Debug.Print "DxfOutputNoScale" = " &  
    swApp.GetUserPreferenceIntegerValue(swDxfOutputNoScale)  
    Debug.Print "DxfMappingFiles" = " &  
    swApp.GetUserPreferenceStringListValue(swDxfMappingFiles)  
    Debug.Print "DxfOutputScaleFactor" = " &  
    swApp.GetUserPreferenceDoubleValue(swDxfOutputScaleFactor)  
    Debug.Print ""  
  
    ' Turn off showing of map  
    bShowMap = swApp.GetUserPreferenceToggle(swDXFDontShowMap)  
    Debug.Print "bShowMap = " & bShowMap  
  
    swApp.SetUserPreferenceToggle swDXFDontShowMap, False  
  
    bRet = swModel.SaveAs4(sPathName, swSaveAsCurrentVersion, swSaveAsOptions_Silent, nErrors,  
nWarnings)  
  
    If bRet = False Then  
        nRetVal = swApp.SendMsgToUser2("Problems saving file.", swMbWarning, swMbOk)  
    End If
```



```
    ' Restore showing of map  
    swApp.SetUserPreferenceToggle swDXFDontShowMap, bShowMap  
End Sub
```

Save Drawing as DXF Example (VB.NET)

This example shows how to save the current drawing file as a DXF file in the same folder.

```
'-----
' Preconditions:
' 1. Open a drawing.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Gets and sets DXF-related system settings.
' 2. Saves the drawing as a DXF file in the same folder as the drawing,
'    overwriting any existing file of the same name.
' 3. Examine the Immediate window and the folder to which the DXF file
'    was saved.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim swModel As ModelDoc2
    Dim sPathName As String
    Dim nErrors As Integer
    Dim nWarnings As Integer
    Dim nRetVal As Integer
    Dim bShowMap As Boolean
    Dim bRet As Boolean

    Sub main()

        swModel = swApp.ActiveDoc
        ' Strip off SOLIDWORKS Design drawing file extension (.slddrw)
        ' and add DXF file extension (.dxf)
        sPathName = swModel.GetPathName
        sPathName = Left(sPathName, Len(sPathName) - 6)
        sPathName = sPathName + ".dxf"

        ' Show current settings

        Debug.Print("DxfMapping = " &
swApp.GetUserPreferenceToggle(swUserPreferenceToggle_e.swDxfMapping))
        Debug.Print("DXFDontShowMap = " &
swApp.GetUserPreferenceToggle(swUserPreferenceToggle_e.swDXFDontShowMap))
        Debug.Print("DxfVersion = " &
swApp.GetUserPreferenceIntegerValue(swUserPreferenceIntegerValue_e.swDxfVersion))
        Debug.Print("DxfOutputFonts = " &
swApp.GetUserPreferenceIntegerValue(swUserPreferenceIntegerValue_e.swDxfOutputFonts))
        Debug.Print("DxfMappingFileIndex = " &
swApp.GetUserPreferenceIntegerValue(swUserPreferenceIntegerValue_e.swDxfMappingFileIndex))
        Debug.Print("DxfOutputLineStyles = " &
swApp.GetUserPreferenceIntegerValue(swUserPreferenceIntegerValue_e.swDxfOutputLineStyles))
        Debug.Print("DxfOutputNoScale = " &
swApp.GetUserPreferenceIntegerValue(swUserPreferenceIntegerValue_e.swDxfOutputNoScale))
        Debug.Print("DxfMappingFiles = " &
swApp.GetUserPreferenceStringListValue(swUserPreferenceStringListValue_e.swDxfMappingFiles))
        Debug.Print("DxfOutputScaleFactor = " &
swApp.GetUserPreferenceDoubleValue(swUserPreferenceDoubleValue_e.swDxfOutputScaleFactor))
        Debug.Print("")

        ' Turn off showing of map
        bShowMap = swApp.GetUserPreferenceToggle(swUserPreferenceToggle_e.swDXFDontShowMap)
        Debug.Print("bShowMap = " & bShowMap)

        swApp.SetUserPreferenceToggle(swUserPreferenceToggle_e.swDXFDontShowMap, False)
```

```
        bRet = swModel.SaveAs4(sPathName, swSaveAsVersion_e.swSaveAsCurrentVersion,  
swSaveAsOptions_e.swSaveAsOptions_Silent, nErrors, nWarnings)  
  
        If bRet = False Then  
            nRetVal = swApp.SendMsgToUser2("Problems saving file.", swMessageBoxIcon_e.swMbWarning,  
swMessageBoxBtn_e.swMbOk)  
        End If  
  
        ' Restore showing of map  
        swApp.SetUserPreferenceToggle(swUserPreferenceToggle_e.swDXFDontShowMap, bShowMap)  
  
    End Sub  
  
    Public swApp As SldWorks  
  
End Class
```

Save Drawing Sheets as DXF Example (VBA)

This example shows how to save multiple drawings sheets as DXF files.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\advdrawings\foodprocessor.slddrw.  
' 2. Verify that C:\temp exists.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Generates a DXF file for each sheet, overwriting any existing file,  
'    and bases the names of the DXF files on the sheet names; for example  
'    Sheet1 is saved as Sheet1.dxf, Sheet2 is saved as Sheet2.dxf, and so on.  
' 2. Examine the Immediate window and C:\temp.  
'-----
```

Option Explicit

```
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swDraw As SldWorks.DrawingDoc  
    Dim vSheetName As Variant  
    Dim nErrors As Long  
    Dim nWarnings As Long  
    Dim nRetVal As Long  
    Dim bShowMap As Boolean  
    Dim nNumSheet As Long  
    Dim i As Long  
    Dim bRet As Boolean  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swDraw = swModel  
  
    ' Current settings  
    Debug.Print "DxfMapping" = " & swApp.GetUserPreferenceToggle(swDxfMapping)  
    Debug.Print "DXFDontShowMap" = " & swApp.GetUserPreferenceToggle(swDXFDontShowMap)  
    Debug.Print "DxfVersion" = " & swApp.GetUserPreferenceIntegerValue(swDxfVersion)  
    Debug.Print "DxfOutputFonts" = " & swApp.GetUserPreferenceIntegerValue(swDxfOutputFonts)  
    Debug.Print "DxfMappingFileIndex" = " & swApp.GetUserPreferenceIntegerValue(swDxfMappingFileIndex)  
    Debug.Print "DxfOutputLineStyle" = " & swApp.GetUserPreferenceIntegerValue(swDxfOutputLineStyle)  
    Debug.Print "DxfOutputNoScale" = " & swApp.GetUserPreferenceIntegerValue(swDxfOutputNoScale)  
    Debug.Print "DxfOutputScaleFactor" = " & swApp.GetUserPreferenceDoubleValue(swDxfOutputScaleFactor)  
    Debug.Print "DxfMappingFiles" = " & swApp.GetUserPreferenceStringListValue(swDxfMappingFiles)  
    Debug.Print ""  
  
    ' Turn off showing of map  
    swApp.SetUserPreferenceToggle swDXFDontShowMap, True  
  
    vSheetName = swDraw.GetSheetNames  
    For i = 0 To UBound(vSheetName)  
        bRet = swDraw.ActivateSheet(vSheetName(i))  
        bRet = swModel.SaveAs4("C:\temp\" & vSheetName(i) & ".dxf", swSaveAsCurrentVersion,  
swSaveAsOptions_Silent, nErrors, nWarnings)  
    Next i  
    ' Switch back to first sheet  
    bRet = swDraw.ActivateSheet(vSheetName(0))  
    ' Restore old setting  
    swApp.SetUserPreferenceToggle swDXFDontShowMap, bShowMap  
  
End Sub
```

Save File as PDF Example (C#)

This example shows how to save multiple drawing sheets to a PDF file.

```
//-----  
// Preconditions:  
// 1. Verify that the specified drawing document to open exists.  
// 2. Verify that c:\test, the folder where to save  
//    the PDF file, exists. If it does not exist,  
//    create it or change the path to a folder that  
//    exists on your system.  
//  
// Postconditions:  
// 1. Opens the specified drawing document.  
// 2. Saves all but the last drawing sheet to an array.  
// 3. Saves the array of drawing sheets to a PDF file  
//    called foodprocessor.pdf.  
// 4. Opens foodprocessor.pdf.  
//-----  
  
using System.Runtime.InteropServices;  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System;  
  
namespace ViewPDF_CSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
        public void Main()  
        {  
            ExportPdfData swExportPDFData = default(ExportPdfData);  
            ModelDoc2 swModel = default(ModelDoc2);  
            ModelDocExtension swModExt = default(ModelDocExtension);  
            DrawingDoc swDrawDoc = default(DrawingDoc);  
            Sheet swSheet = default(Sheet);  
            bool boolstatus = false;  
            string filename = null;  
            int errors = 0;  
            int warnings = 0;  
            string[] obj = null;  
  
            // Save PDF file to this folder and filename  
            filename = "c:\\test\\foodprocessor.pdf";  
  
            // Open drawing document  
            swModel =  
(ModelDoc2)swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS  
2018\\samples\\tutorial\\advdrawings\\foodprocessor.slddrw", (int)swDocumentTypes_e.swDocDRAWING,  
(int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);  
            swModExt = (ModelDocExtension)swModel.Extension;  
            swExportPDFData =  
(ExportPdfData)swApp.GetExportFileData((int)swExportDataFileType_e.swExportPdfData);  
  
            // Get the names of the drawing sheets in the drawing  
            // to get the size of the array of drawing sheets  
            swDrawDoc = (DrawingDoc)swModel;  
  
            obj = (string[])swDrawDoc.GetSheetNames();  
            int count = 0;  
            count = obj.Length;  
            int i = 0;  
            object[] objs = new object[count - 1];  
            Dispatcher[] arrObjIn = new Dispatcher[count - 1];
```

```

// Activate each drawing sheet, except the last drawing sheet, for
// demonstration purposes only and add each sheet to an array
// of drawing sheets
for (i = 0; i < count - 1; i++)
{
    boolstatus = swDrawDoc.ActivateSheet((obj[i]));
    swSheet = (Sheet)swDrawDoc.GetCurrentSheet();
    objs[i] = swSheet;
    arrObjIn[i] = new DispatcherWrapper(objs[i]);

}

// Save the drawings sheets to a PDF file
boolstatus =
swExportPDFData.SetSheets((int)swExportDataSheetsToExport_e.swExportData_ExportSpecifiedSheets,
(arrObjIn));
    swExportPDFData.ViewPdfAfterSaving = true;
    boolstatus = swModExt.SaveAs(filename, (int)swSaveAsVersion_e.swSaveAsCurrentVersion,
(int)swSaveAsOptions_e.swSaveAsOptions_Silent, swExportPDFData, ref errors, ref warnings);
}

    public SldWorks swApp;

}

}

```

Save File as PDF Example (VBA)

This example shows how to export the specified sheet in a drawing document to a PDF file.

```
'-----  
' Preconditions:  
' 1. Verify that the specified drawing document to open exists.  
' 2. Verify that c:\test, the folder where to save  
'    the PDF file, exists. If it does not exist,  
'    create it or change the path to a folder that  
'    exists on your system.  
'  
' Postconditions:  
' 1. Opens the specified drawing document.  
' 2. Saves all but the last drawing sheet to an array.  
' 3. Saves the array of drawing sheets to a PDF file  
'    called foodprocessor.pdf.  
' 4. Opens foodprocessor.pdf.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swExportPDFData As SldWorks.ExportPdfData  
Dim boolstatus As Boolean  
Dim filename As String  
Dim lErrors As Long  
Dim lWarnings As Long  
Dim strSheetName(4) As String  
Dim varSheetName As Variant
```

Sub main()

```
    ' Path to which to save PDF file of drawing  
    filename = "C:\test\foodprocessor.PDF"  
  
    Set swApp = Application.SldWorks  
    swApp.Visible = True  
  
    ' Open specified drawing  
    Set swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS  
2018\samples\tutorial\advdrawings\foodprocessor.slddrw", swDocDRAWING, swOpenDocOptions_Silent,  
    "", lErrors, lWarnings)  
    Set swModelDocExt = swModel.Extension  
    Set swExportPDFData = swApp.GetExportFileData(1)  
  
    ' Names of the sheets  
    strSheetName(0) = "Sheet1"  
    strSheetName(1) = "Sheet2"  
    strSheetName(2) = "Sheet3"  
    ' strSheetName(3) = "Sheet4"  
  
    varSheetName = strSheetName  
  
    If swExportPDFData Is Nothing Then MsgBox "Nothing"  
    boolstatus = swExportPDFData.SetSheets(swExportData_ExportSpecifiedSheets, varSheetName)  
    swExportPDFData.ViewPdfAfterSaving = True  
  
    boolstatus = swModelDocExt.SaveAs(filename, 0, 0, swExportPDFData, lErrors, lWarnings)  
End Sub
```

Save File as PDF Example (VB.NET)

This example shows how to save multiple drawing sheets to a PDF file.

```
'-----  
' Preconditions:  
' 1. Verify that the specified drawing document to open exists.  
' 2. Verify that c:\test, the folder where to save  
'    the PDF file, exists. If it does not exist,  
'    create it or change the path to a folder that  
'    exists on your system.  
'  
' Postconditions:  
' 1. Opens the specified drawing document.  
' 2. Saves all but the last drawing sheet to an array.  
' 3. Saves the array of drawing sheets to a PDF file  
'    called foodprocessor.pdf.  
' 4. Opens foodprocessor.pdf.  
'-----
```

```
Imports System.Runtime.InteropServices  
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System  
Imports System.Diagnostics
```

Partial Class SolidWorksMacro

Public Sub main()

```
    Dim swExportPDFData As ExportPdfData  
    Dim swModel As ModelDoc2  
    Dim swModExt As ModelDocExtension  
    Dim swDrawDoc As DrawingDoc  
    Dim boolstatus As Boolean  
    Dim filename As String  
    Dim errors As Integer  
    Dim warnings As Integer  
    Dim objs() As Object  
    Dim obj As Object  
  
    ' Save PDF file to this folder and filename  
    filename = "c:\test\foodprocessor.pdf"  
  
    ' Open drawing document  
    swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS  
2018\samples\tutorial\advdrawings\foodprocessor.slddrw", swDocumentTypes_e.swDocDRAWING,  
swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)  
    swModExt = swModel.Extension  
    swExportPDFData = swApp.GetExportFileData(swExportDataFileType_e.swExportPdfData)  
  
    ' Get the names of the drawing sheets in the drawing  
    ' to get the size of the array of drawing sheets  
    swDrawDoc = swModel  
    obj = swDrawDoc.GetSheetNames  
  
    Dim count As Integer  
    count = UBound(obj)  
    ReDim objs(count)  
    Dim arrObjIn(count) As DispatchWrapper  
  
    Dim i As Integer  
    ' Activate each drawing sheet, except the last drawing sheet, for  
    ' demonstration purposes only and add each sheet to an array
```



```

' of drawing sheets
For i = 0 To count - 1
    boolstatus = swDrawDoc.ActivateSheet(obj(i))
    Dim swSheet As Sheet
    swSheet = swDrawDoc.GetCurrentSheet
    objs(i) = swSheet
    arrObjIn(i) = New DispatcherWrapper(objs(i))
Next i

' Save the drawings sheets to a PDF file
boolstatus =
swExportPDFData.SetSheets(swExportDataSheetsToExport_e.swExportData_ExportSpecifiedSheets,
(arrObjIn))
swExportPDFData.ViewPdfAfterSaving = True
Debug.Print(swExportPDFData.ViewPdfAfterSaving)
boolstatus = swModExt.SaveAs(filename, swSaveAsVersion_e.swSaveAsCurrentVersion,
swSaveAsOptions_e.swSaveAsOptions_Silent, swExportPDFData, errors, warnings)

End Sub

Public swApp As SldWorks

End Class

```

Save File Example (C#)

This example shows how to save a file.

```
//-----  
// Preconditions:  
// 1. Open a model.  
// 2. Open the Immediate window.  
//  
// Postconditions:  
// 1. Zooms to fit the model in the graphics area.  
// 2. Saves the model.  
// 3. Examine the graphics area and Immediate window.  
//-----  
  
using Microsoft.VisualBasic;  
  
using System;  
  
using System.Collections;  
  
using System.Collections.Generic;  
  
using System.Data;  
  
using System.Diagnostics;  
  
using SolidWorks.Interop.sldworks;  
  
using SolidWorks.Interop.swconst;  
  
using System.Runtime.InteropServices;  
  
namespace SaveFile_CSharp.csproj  
{  
  
    partial class SolidWorksMacro  
    {  
  
        ModelDoc2 swModel;  
  
        bool boolstatus;  
  
        int lErrors;  
  
  
        int lWarnings;  
  
        public void Main()  
        {  
  
            swModel = (ModelDoc2)swApp.ActiveDoc;  
  
            swApp.Visible = true;  
  
            // Make a change  
  
            swModel.ViewZoomtofit2();  
  
  
            boolstatus = swModel.Save3((int)swSaveAsOptions_e.swSaveAsOptions_Silent, ref lErrors, ref  
lWarnings);
```

```
        // Errors
        Debug.Print("Errors as defined in swFileSaveError_e: " + lErrors);

        // Warnings
        Debug.Print("Warnings as defined in swFileSaveWarning_e: " + lWarnings);

    }

    public SldWorks swApp;

}

}
```

Save File Example (VBA)

This example shows how to save a file.

```
'-----  
' Preconditions:  
' 1. Open a model.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Zooms to fit the model in the graphics area.  
' 2. Saves the model.  
' 3. Examine the graphics area and Immediate window.  
'-----  
Option Explicit  
  
    Dim swApp                As SldWorks.SldWorks  
    Dim swModel              As SldWorks.ModelDoc2  
    Dim boolstatus           As Boolean  
    Dim lErrors              As Long  
    Dim lWarnings            As Long  
  
Sub main()  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    swApp.Visible = True  
  
    ' Make a change  
    swModel.ViewZoomtofit2  
  
    boolstatus = swModel.Save3(swSaveAsOptions_Silent, lErrors, lWarnings)  
  
    ' Errors  
    Debug.Print ("Errors as defined in swFileSaveError_e: " & lErrors)  
  
    ' Warnings  
    Debug.Print ("Warnings as defined in swFileSaveWarning_e: " & lWarnings)  
End Sub
```

Save File Example (VB.NET)

This example shows how to save a file.

```
'-----  
' Preconditions:  
' 1. Open a model.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Zooms to fit the model in the graphics area.  
' 2. Saves the model.  
' 3. Examine the graphics area and Immediate window.  
'-----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System.Runtime.InteropServices
```

```
Imports System
```

```
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Dim swModel As ModelDoc2
```

```
    Dim boolstatus As Boolean
```

```
    Dim lErrors As Integer
```

```
    Dim lWarnings As Integer
```

```
    Sub main()
```

```
        swModel = swApp.ActiveDoc
```

```
        swApp.Visible = True
```

```
        '  
        Make a change
```

```
        swModel.ViewZoomtofit2()
```

```
        boolstatus = swModel.Save3(swSaveAsOptions_e.swSaveAsOptions_Silent, lErrors, lWarnings)
```

```
        ' Errors
```

```
        Debug.Print("Errors as defined in swFileSaveError_e: " & lErrors)
```

```
        ' Warnings
```

```
        Debug.Print("Warnings as defined in swFileSaveWarning_e: " & lWarnings)
```

```
    End Sub
```

```
Public swApp As SldWorks
```

```
End Class
```

Save Model as BMP Example (C#)

This example shows how to save the current view of a model as a bitmap.

```
//-----  
// Preconditions:  
// 1. Open:  
//   C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\cstick.sldprt  
// 2. Run macro.  
//  
// Postconditions: Current view of the model is saved as C:\cstick.bmp.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
using System;  
  
namespace SaveBMPCSharp.csproj  
{  
  
    partial class SolidWorksMacro  
    {  
  
        public void Main()  
        {  
  
            ModelDoc2 swModel = default(ModelDoc2);  
            bool returnVal = false;  
            string fileName = null;  
  
            swModel = (ModelDoc2)swApp.ActiveDoc;  
            fileName = "c:\\cstick.bmp";  
  
            // Save as bitmap and use current window size  
            returnVal = swModel.SaveBMP(fileName, 0, 0);  
  
        }  
  
        /// <summary>  
        /// The SldWorks swApp variable is pre-assigned for you.  
        /// </summary>  
        public SldWorks swApp;  
  
    }  
}
```

Save Model as Bitmap Example (VBA)

This example shows how to save the current view of a model as a bitmap.

```
'-----  
' Preconditions:  
' 1. Open:  
'    C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\cstick.sldprt  
' 2. Run macro.  
'  
' Postconditions: Current view of the model is saved as C:\cstick.bmp.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim returnVal As Boolean  
Dim fileName As String  
  
Sub main()  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    fileName = "c:\cstick.bmp"  
  
    ' Save as bitmap and use current window size  
    returnVal = swModel.SaveBMP(fileName, 0, 0)  
  
End Sub
```


Save_Model_as_Bitmap_Example_VBNET (Topic)

SOLIDWORKS API Help

Save Model as Bitmap Example (VB.NET)

This example shows how to save the current view of a model as a bitmap.

```
'-----  
' Preconditions:  
' 1. Open:  
'   C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\cstick.sldprt  
' 2. Run macro.  
'  
' Postconditions: Current view of the model is saved as C:\cstick.bmp.  
'-----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System
```

Partial Class SolidWorksMacro

Public Sub Main()

```
    Dim swModel As ModelDoc2  
    Dim returnVal As Boolean  
    Dim fileName As String
```

```
    swModel = swApp.ActiveDoc  
    fileName = "c:\cstick.bmp"
```

```
    ' Save as bitmap and use current window size  
    returnVal = swModel.SaveBMP(fileName, 0, 0)
```

End Sub

```
''' <summary>  
''' The SldWorks swApp variable is pre-assigned for you.  
''' </summary>  
Public swApp As SldWorks
```

End Class

Save Rolled Backed Part as Parasolid File Example (VBA)

This example shows how to save a part in a rolled backed state as a Parasolid file.

NOTE: When a part is in a rolled back state, you cannot export the part to any other format until the model has been rolled forward. This example shows how to use various geometry-related and topology-related method to create a temporary part that you can then export to any other format, in this example, Parasolid.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\api\box.sldprt.  
' 2. Drag the rollback bar to just below Boss-Extrude1.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Saves the Parasolid file to the same folder as the part,  
'    overwriting any existing file of the same name; the  
'    Parasolid file has same geometry as the part in the rolled back  
'    state.  
' 2. Examine the Immediate window.  
' 3. Open public_documents\samples\tutorial\api\box.x_t.  
'  
' NOTE: Because the part is used elsewhere, do not save changes.  
'-----
```

Option Explicit

Sub main()

```
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swPart As SldWorks.PartDoc  
    Dim swNewPart As SldWorks.PartDoc  
    Dim swNewModel As SldWorks.ModelDoc2  
    Dim sNewModelName As String  
    Dim vBodyArr As Variant  
    Dim vBody As Variant  
    Dim swBody As SldWorks.Body2  
    Dim swCopyBody As SldWorks.Body2  
    Dim swFeat As SldWorks.Feature  
    Dim sPathName As String  
    Dim nOldVal As Long  
    Dim nErrors As Long  
    Dim nWarnings As Long  
    Dim nRetVal As Long  
    Dim i As Long  
    Dim bRet As Boolean  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swPart = swModel  
    Set swNewPart = swApp.NewPart  
    Set swNewModel = swNewPart  
  
    sNewModelName = swNewModel.GetTitle  
  
    Debug.Print "Parasolid settings:"  
  
    ' Current Parasolid settings  
    Debug.Print "  Current:"  
    Debug.Print "    swUserPreferenceIntegerValue_e.ParasolidOutputVersion (0 =  
swParasolidOutputVersion_e.swParasolidOutputVersion_latest) = " +  
Str(swApp.GetUserPreferenceIntegerValue(swParasolidOutputVersion))  
    Debug.Print "    swUserPreferenceStringValue_e.SaveAsCoordinateSystem (empty string signifies that the default  
coordinate system that does not require a transform) = " +  
swModel.GetUserPreferenceStringValue(swFileSaveAsCoordinateSystem)  
  
    ' Strip off SOLIDWORKS Design file extension (sldxxx)  
    ' and add Parasolid extension (x_b)  
    sPathName = swModel.GetPathName  
    sPathName = Left(sPathName, Len(sPathName) - 6)
```

```

sPathName = sPathName + "x_t"

' Store existing Parasolid export version and change to v10.0
nOldVal = swApp.GetUserPreferenceIntegerValue(swParasolidOutputVersion)
bRet = swApp.SetUserPreferenceIntegerValue(swParasolidOutputVersion, swParasolidOutputVersion_100)
Debug.Print "    Modified: "
Debug.Print "        swUserPreferenceIntegerValue_e.ParasolidOutputVersion (4 =
swParasolidOutputVersion_e.swParasolidOutputVersion_100) = " +
Str(swApp.GetUserPreferenceIntegerValue(swParasolidOutputVersion))

' Create temporary part with bodies from existing part
For i = 0 To 5
    vBodyArr = swPart.GetBodies2(i, False)
    If Not IsEmpty(vBodyArr) Then
        For Each vBody In vBodyArr
            Set swBody = vBody
            Set swCopyBody = swBody.Copy
            Set swFeat = swNewPart.CreateFeatureFromBody3(swCopyBody, False, swCreateFeatureBodyCheck +
swCreateFeatureBodySimplify)
        Next vBody
    End If
Next i

' Export temporary part to Parasolid
bRet = swNewModel.SaveAs4(sPathName, swSaveAsCurrentVersion, swSaveAsOptions_Silent, nErrors, nWarnings)

If bRet = False Then
    nRetVal = swApp.SendMsgToUser2("Problems saving file.", swMbWarning, swMbOk)
End If

' Get rid of temporary file
swApp.QuitDoc sNewModelName

' Restore old Parasolid export version
bRet = swApp.SetUserPreferenceIntegerValue(swParasolidOutputVersion, nOldVal)
Debug.Print "    Restored:"
Debug.Print "        swUserPreferenceIntegerValue_e.ParasolidOutputVersion (0 =
swParasolidOutputVersion_e.swParasolidOutputVersion_latest) = " +
Str(swApp.GetUserPreferenceIntegerValue(swParasolidOutputVersion))

End Sub

```

Save Solid Body to File Example (C#)

This example shows how to save a weldment member to another part document.

```
//-----  
// Preconditions: Open public_documents\samples\tutorial\api\weldment_box3.sldprt.  
//  
// Postconditions:  
// 1. Updates the cut list in the weldment part.  
// 2. Saves the first weldment member in the FeatureManager design tree to  
//    RefWeldment1.sldprt, and saves the cut list properties in the parent part  
//    to the cut list of the new part.  
// 3. Opens RefWeldment1.sldprt and updates its cut list. (This step could  
//    take a few minutes to complete.)  
// 4. At System.Diagnostics.Debugger.Break(), press F5.  
// 5. Closes RefWeldment1.sldprt.  
// 6. To verify steps 2 and 3, open and inspect RefWeldment1.sldprt, which resides  
//    in the same folder as this macro.  
//  
// NOTE: Because weldment_box3.sldprt is used elsewhere, do not save changes.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
using System;  
namespace CreatePartForSolidBody_CSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
        Feature swFeat;  
        BodyFolder swBodyFolder;  
        bool updateBoolstatus;  
        bool boolstatus;  
        int longstatus;  
        int longWarnings;  
        ModelDoc2 currentModel;  
        ModelDoc2 swModel;  
        int modelType;  
        string modelTitle;  
  
        public void Main()  
        {  
            currentModel = (ModelDoc2)swApp.ActiveDoc;  
  
            modelTitle = currentModel.GetTitle();  
            modelType = currentModel.GetType();  
  
            swFeat = (Feature)currentModel.FirstFeature();  
            if (swFeat == null)  
                ErrorMsg(swApp, "Failed to get first feature");  
  
            while ((swFeat != null))  
            {  
                if (swFeat.GetTypeName2() == "SolidBodyFolder")  
                {  
                    swBodyFolder = (BodyFolder)swFeat.GetSpecificFeature2();  
                    if (swBodyFolder == null)  
                        ErrorMsg(swApp, "Failed to get body folder");  
  
                    boolstatus = swBodyFolder.SetAutomaticCutList(true);  
  
                    boolstatus = swBodyFolder.UpdateCutList();  
  
                    break;  
                }  
            }  
        }  
    }  
}
```

```

    }
    swFeat = (Feature)swFeat.GetNextFeature();
}

updateBoolstatus = false;

swFeat = (Feature)currentModel.FirstFeature();
if (swFeat == null)
    ErrorMsg(swApp, "Failed to get first feature");

while ((swFeat != null))
{
    if (swFeat.GetTypeName2() == "WeldMemberFeat")
    {
        boolstatus = swFeat.Select2(false, 0);
        if (boolstatus == false)
            ErrorMsg(swApp, "Failed to select feature");

        // Save the selected solid body weldment member to another part,
        // transferring the solid body's cut list properties to the new part's cut
list;

        // automatically creates a weldment and cut list folder
        boolstatus =
((PartDoc)currentModel).SaveToFile3(swApp.GetCurrentMacroPathFolder() + "\\RefWeldment1" +
".sldprt", 1, swCutListTransferOptions_e.swCutListTransferOptions_CutListProperties, false, "",
out longstatus, out longWarnings);

        System.Diagnostics.Debugger.Break();

        if (boolstatus == false)
            ErrorMsg(swApp, "Failed to insert weldment member into new part");

        swModel = (ModelDoc2)swApp.ActiveDoc;
        if (swModel == null)
            ErrorMsg(swApp, "Failed to set open model as active document");

        updateBoolstatus = true;
        break;
    }
    swFeat = (Feature)swFeat.GetNextFeature();
}

if (updateBoolstatus == true)
{
    swFeat = (Feature)currentModel.FirstFeature();
    if (swFeat == null)
        ErrorMsg(swApp, "Failed to get first feature");

    while ((swFeat != null))
    {
        if (swFeat.GetTypeName2() == "SolidBodyFolder")
        {
            swBodyFolder = (BodyFolder)swFeat.GetSpecificFeature2();
            if (swBodyFolder == null)
                ErrorMsg(swApp, "Failed to get body folder");

            boolstatus = swBodyFolder.SetAutomaticCutList(true);
            if (boolstatus == false)
                ErrorMsg(swApp, "Failed to set cut list to automatic");

            boolstatus = swBodyFolder.UpdateCutList();
            if (boolstatus == false)
                ErrorMsg(swApp, "Failed to update cut list");

            swApp.CloseDoc(swModel.GetTitle());
            break;
        }
        swFeat = (Feature)swFeat.GetNextFeature();
    }
}
}
}

```

```
public void ErrorMsg(SldWorks Swapp, string Message)
{
    Swapp.SendMsgToUser2(Message, 0, 0);
    Swapp.RecordLine("*** WARNING - General");
    Swapp.RecordLine("*** " + Message);
    Swapp.RecordLine("");
}
```

```
public SldWorks swApp;
```

```
}
```

```
}
```

Save Solid Body to File Example (VBA)

This example shows how to save a weldment member to another part document.

```
'-----  
' Preconditions: Open public_documents\samples\tutorial\api\weldment_box3.sldprt.  
'  
' Postconditions:  
' 1. Updates the cut list in the weldment part.  
' 2. Saves the first weldment member in the FeatureManager design tree to  
'   RefWeldment1.sldprt, and saves the cut list properties in the parent part  
'   to the cut list of the new part.  
' 3. Opens RefWeldment1.sldprt and updates its cut list. (This step could  
'   take a few minutes to complete.)  
' 4. At Stop, press F5.  
' 5. Closes RefWeldment1.sldprt.  
' 6. To verify steps 2 and 3, open and inspect RefWeldment1.sldprt, which resides in  
'   the same folder as this macro.  
'  
' NOTE: Because weldment_box3.sldprt is used elsewhere, do not save changes.  
'-----  
Dim Swapp As SldWorks.SldWorks  
  
Dim swFeat As SldWorks.Feature  
  
Dim swBodyFolder As SldWorks.BodyFolder  
  
Dim updateBoolstatus As Boolean  
  
Dim boolstatus As Boolean  
  
Dim longstatus As Long  
  
Dim longWarnings As Long  
  
Dim currentModel As SldWorks.ModelDoc2  
  
Dim swModel As SldWorks.ModelDoc2  
  
Dim modelType As Long  
  
Dim modelTitle As String  
  
Option Explicit  
Sub main()  
    Set Swapp = Application.SldWorks  
    Set currentModel = Swapp.ActiveDoc  
  
    modelTitle = currentModel.GetTitle  
    modelType = currentModel.GetType  
  
    Set swFeat = currentModel.FirstFeature  
    If swFeat Is Nothing Then MsgBox Swapp, "Failed to get first feature"  
  
    Do While Not swFeat Is Nothing  
        If swFeat.GetTypeName2 = "SolidBodyFolder" Then  
            Set swBodyFolder = swFeat.GetSpecificFeature2  
            If swBodyFolder Is Nothing Then MsgBox Swapp, "Failed to get body folder"  
  
            boolstatus = swBodyFolder.SetAutomaticCutList(True)  
  
            boolstatus = swBodyFolder.UpdateCutList()  
        End If  
    End Do  
End Sub
```

```

        Exit Do
    End If
    Set swFeat = swFeat.GetNextFeature
Loop

updateBoolstatus = False

Set swFeat = currentModel.FirstFeature
If swFeat Is Nothing Then ErrorMessage Swapp, "Failed to get first feature"

Do While Not swFeat Is Nothing
    If swFeat.GetType = "WeldMemberFeat" Then
        boolstatus = swFeat.Select2(False, 0)
        If boolstatus = False Then ErrorMessage Swapp, "Failed to select feature"

        ' Save the selected solid body weldment member to another part,
        ' transferring the solid body's cut list properties to the new part's cut list;
        ' automatically creates a weldment and cut list folder
        boolstatus = currentModel.SaveToFile3(Swapp.GetCurrentMacroPathFolder +
"\RefWeldment1" + ".sldprt", 1, swCutListTransferOptions_CutListProperties, False, "", longstatus,
longWarnings)
        Stop
        If boolstatus = False Then ErrorMessage Swapp, "Failed to insert weldment member into new
part"

        Set swModel = Swapp.ActiveDoc
        If swModel Is Nothing Then ErrorMessage Swapp, "Failed to set open model as active
document"

        updateBoolstatus = True
        Exit Do
    End If
    Set swFeat = swFeat.GetNextFeature
Loop

If updateBoolstatus = True Then
    Set swFeat = currentModel.FirstFeature
    If swFeat Is Nothing Then ErrorMessage Swapp, "Failed to get first feature"

    Do While Not swFeat Is Nothing
        If swFeat.GetType = "SolidBodyFolder" Then
            Set swBodyFolder = swFeat.GetSpecificFeature2
            If swBodyFolder Is Nothing Then ErrorMessage Swapp, "Failed to get body folder"

            boolstatus = swBodyFolder.SetAutomaticCutList(True)
            If boolstatus = False Then ErrorMessage Swapp, "Failed to set cut list to automatic"

            boolstatus = swBodyFolder.UpdateCutList()
            If boolstatus = False Then ErrorMessage Swapp, "Failed to update cut list"

            Swapp.CloseDoc swModel.GetTitle
            Exit Do
        End If
        Set swFeat = swFeat.GetNextFeature
    Loop
End If
End Sub

Function ErrorMessage(Swapp As Object, Message As String)
    Swapp.SendMsgToUser2 Message, 0, 0
    Swapp.RecordLine "**** WARNING - General"
    Swapp.RecordLine "**** " & Message
    Swapp.RecordLine ""
End Function

```


Save Solid Body to File Example (VB.NET)

This example shows how to save a weldment member to another part document.

```
'-----  
' Preconditions: Open public_documents\samples\tutorial\api\weldment_box3.sldprt.  
'  
' Postconditions:  
' 1. Updates the cut list in the weldment part.  
' 2. Saves the first weldment member in the FeatureManager design tree to  
'    RefWeldment1.sldprt, and saves the cut list properties in the parent part  
'    to the cut list of the new part.  
' 3. Opens RefWeldment1.sldprt and updates its cut list. (This step could  
'    take a few minutes to complete.)  
' 4. At Stop, press F5.  
' 5. Closes RefWeldment1.sldprt.  
' 6. To verify steps 2 and 3, open and inspect RefWeldment1.sldprt, which resides  
'    in the same folder as this macro.  
'  
' NOTE: Because weldment_box3.sldprt is used elsewhere, do not save changes.  
'-----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Dim swFeat As Feature  
    Dim swBodyFolder As BodyFolder  
    Dim updateBoolstatus As Boolean  
    Dim boolstatus As Boolean  
    Dim longstatus As Integer  
    Dim longWarnings As Integer  
    Dim currentModel As ModelDoc2  
    Dim swModel As ModelDoc2  
    Dim modelType As Integer  
    Dim modelTitle As String
```

```
    Sub main()  
  
        currentModel = swApp.ActiveDoc  
  
        modelTitle = currentModel.GetTitle  
        modelType = currentModel.GetType  
  
        swFeat = currentModel.FirstFeature  
        If swFeat Is Nothing Then ErrorMsg(swApp, "Failed to get first feature")  
  
        Do While Not swFeat Is Nothing  
            If swFeat.GetTypeName2 = "SolidBodyFolder" Then  
                swBodyFolder = swFeat.GetSpecificFeature2  
                If swBodyFolder Is Nothing Then ErrorMsg(swApp, "Failed to get body folder")  
  
                boolstatus = swBodyFolder.SetAutomaticCutList(True)  
  
                boolstatus = swBodyFolder.UpdateCutList()  
  
                Exit Do  
            End If  
            swFeat = swFeat.GetNextFeature  
        Loop  
  
        updateBoolstatus = False
```

```

swFeat = currentModel.FirstFeature
If swFeat Is Nothing Then ErrorMsg(swApp, "Failed to get first feature")

Do While Not swFeat Is Nothing
    If swFeat.GetTypeName2 = "WeldMemberFeat" Then
        boolstatus = swFeat.Select2(False, 0)
        If boolstatus = False Then ErrorMsg(swApp, "Failed to select feature")

        ' Save the selected solid body weldment member to another part,
        ' transferring the solid body's cut list properties to the new part's cut list;
        ' automatically creates a weldment and cut list folder
        boolstatus = currentModel.SaveToFile3(swApp.GetCurrentMacroPathFolder +
"\RefWeldment1" + ".sldprt", 1,
swCutListTransferOptions_e.swCutListTransferOptions_CutListProperties, False, "", longstatus,
longWarnings)

        Stop

        If boolstatus = False Then ErrorMsg(swApp, "Failed to insert weldment member into
new part")

        swModel = swApp.ActiveDoc
        If swModel Is Nothing Then ErrorMsg(swApp, "Failed to set open model as active
document")

        updateBoolstatus = True
        Exit Do
    End If
    swFeat = swFeat.GetNextFeature
Loop

If updateBoolstatus = True Then
    swFeat = currentModel.FirstFeature
    If swFeat Is Nothing Then ErrorMsg(swApp, "Failed to get first feature")

    Do While Not swFeat Is Nothing
        If swFeat.GetTypeName2 = "SolidBodyFolder" Then
            swBodyFolder = swFeat.GetSpecificFeature2
            If swBodyFolder Is Nothing Then ErrorMsg(swApp, "Failed to get body folder")

            boolstatus = swBodyFolder.SetAutomaticCutList(True)
            If boolstatus = False Then ErrorMsg(swApp, "Failed to set cut list to
automatic")

            boolstatus = swBodyFolder.UpdateCutList()
            If boolstatus = False Then ErrorMsg(swApp, "Failed to update cut list")

            swApp.CloseDoc(swModel.GetTitle)
            Exit Do
        End If
        swFeat = swFeat.GetNextFeature
    Loop
End If

End Sub

Sub ErrorMsg(ByVal Swapp As Object, ByVal Message As String)
    Swapp.SendMsgToUser2(Message, 0, 0)
    Swapp.RecordLine("**** WARNING - General")
    Swapp.RecordLine("**** " & Message)
    Swapp.RecordLine("")
End Sub

Public swApp As SldWorks

End Class

```

Save Table to PDF Example (C#)

This example shows how to save a table as a PDF file.

```
//-----  
// Preconditions:  
// 1. Open a part document that contains a table annotation.  
// 2. Select the table annotation.  
//  
// Postconditions:  
// 1. Saves the selected table as c:\test\table.pdf.  
// 2. Examine c:\test\table.pdf.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
using System;  
  
namespace SaveAsPDFCSharp.csproj  
{  
    public partial class SolidWorksMacro  
    {  
  
        public void Main()  
        {  
            ModelDoc2 swModel = default(ModelDoc2);  
            SelectionMgr swSelectionMgr = default(SelectionMgr);  
            TableAnnotation swTableAnnotation = default(TableAnnotation);  
  
            swModel = (ModelDoc2)swApp.ActiveDoc;  
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;  
  
            //Get the selected table and save it as a PDF file  
            swTableAnnotation = (TableAnnotation)swSelectionMgr.GetSelectedObject6(1, 0);  
            swTableAnnotation.SaveAsPDF("c:\\test\\table.pdf");  
  
        }  
  
        /// <summary>  
        /// The SldWorks swApp variable is pre-assigned for you.  
        /// </summary>  
        public SldWorks swApp;  
    }  
}
```

Save Table to PDF Example (VBA)

This example shows how to save a table as a PDF file.

```
'-----  
' Preconditions:  
' 1. Open a part document that contains a table annotation.  
' 2. Select the table annotation.  
'  
' Postconditions:  
' 1. Saves the selected table as c:\test\table.pdf.  
' 2. Examine c:\test\table.pdf.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swSelectionMgr As SldWorks.SelectionMgr  
Dim swTableAnnotation As SldWorks.TableAnnotation  
  
Sub main()  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelectionMgr = swModel.SelectionManager  
  
    'Get the selected table and save as a PDF file  
    Set swTableAnnotation = swSelectionMgr.GetSelectedObject6(1, 0)  
    swTableAnnotation.SaveAsPDF ("c:\test\table.pdf")  
  
End Sub
```

Save Table to PDF Example (VB.NET)

This example shows how to save a table as a PDF file.

```
'-----  
' Preconditions:  
' 1. Open a part document that contains a table annotation.  
' 2. Select the table annotation.  
'  
' Postconditions:  
' 1. Saves the selected table as c:\test\table.pdf.  
' 2. Examine c:\test\table.pdf.  
'-----  
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System  
  
Partial Class SolidWorksMacro  
  
    Public Sub Main()  
  
        Dim swModel As ModelDoc2  
        Dim swSelectionMgr As SelectionMgr  
        Dim swTableAnnotation As TableAnnotation  
  
        swModel = swApp.ActiveDoc  
        swSelectionMgr = swModel.SelectionManager  
  
        'Get the selected table and save as a PDF file  
        swTableAnnotation = swSelectionMgr.GetSelectedObject6(1, 0)  
        swTableAnnotation.SaveAsPDF("c:\test\table.pdf")  
  
    End Sub  
  
    ''' <summary>  
    ''' The SldWorks swApp variable is pre-assigned for you.  
    ''' </summary>  
    Public swApp As SldWorks  
  
End Class
```

Save Table to Microsoft Excel Example (VBA)

This example shows how to save a BOM table annotation to a Microsoft Excel document.

```
' -----  
' Preconditions:  
' 1. Open a drawing of an assembly with a BOM.  
' 2. Click the move-table icon in the upper-left corner  
'    of the BOM table to open the table's PropertyManager page.  
'  
' Postconditions: Saves the selected BOM to c:\temp\BOMTable.xls.  
' -----  
  
Dim swApp As SldWorks.SldWorks  
Dim swModDoc As SldWorks.IModelDoc2  
Dim swTable As SldWorks.ITableAnnotation  
  
Option Explicit  
  
Public Sub Main()  
    Set swApp = Application.SldWorks  
  
    Set swModDoc = swApp.ActiveDoc  
    Dim swSM As ISelectionMgr  
    Set swSM = swModDoc.SelectionManager  
    Set swTable = swSM.GetSelectedObject6(1, -1)  
    swModDoc.ClearSelection2 (True)  
  
    Dim swSpecTable As IBomTableAnnotation  
    Set swSpecTable = swTable  
  
    ' Save the selected BOM table to Microsoft Excel, including hidden cells  
    ' and images  
    status = swSpecTable.SaveAsExcel("c:\temp\BOMTable.xls", True, True)  
  
End Sub
```

Scale Hatch Pattern to Section View Example (C#)

This example shows how to scale a hatch pattern to a section view.

```
//-----  
// Preconditions:  
// 1. Open public_documents\introsw\bolt-assembly.slddrw.  
// 2. Open the Immediate window.  
//  
// Postconditions:  
// 1. Scales the hatch pattern to the section view.  
// 2. Examine the Immediate window.  
//  
// NOTE: Because this drawing is used elsewhere, do not save changes.  
//-----  
using System;  
using System.Diagnostics;  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
namespace Macro1CSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
        ModelDoc2 swModel;  
        DrawingDoc swDrawing;  
        View swView;  
        DrSection swSectionView;  
  
        public void Main()  
        {  
            swModel = (ModelDoc2)swApp.ActiveDoc;  
            swDrawing = (DrawingDoc)swModel;  
            swDrawing.ActivateView("Drawing View5");  
            swView = (View)swDrawing.ActiveDrawingView;  
            swSectionView = (DrSection)swView.GetSection();  
            swSectionView.ScaleHatchPattern = true;  
            Debug.Print(" Scale hatch pattern to section view? " + swSectionView.ScaleHatchPattern);  
        }  
  
        /// <summary>  
        /// The SldWorks swApp variable is pre-assigned for you.  
        /// </summary>  
  
        public SldWorks swApp;  
    }  
}
```

Scale Hatch Pattern to Section View Example (VBA)

This example shows how to scale a hatch pattern to a section view.

```
'-----  
' Preconditions:  
' 1. Open public_documents\introsw\bolt-assembly.slddrw.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Scales the hatch pattern to the section view.  
' 2. Examine the Immediate window.  
'  
' NOTE: Because this drawing is used elsewhere, do not save changes.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swDrawing As SldWorks.DrawingDoc  
Dim swView As SldWorks.View  
Dim swSectionView As SldWorks.DrSection
```

Sub main()

```
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swDrawing = swModel  
  
    swDrawing.ActivateView("Drawing View5")  
    Set swView = swDrawing.ActiveDrawingView  
    Set swSectionView = swView.GetSection  
    swSectionView.ScaleHatchPattern = True  
    Debug.Print "  Scale hatch pattern to section view? " & swSectionView.ScaleHatchPattern
```

End Sub

Scale_Hatch_Pattern_With_Section_View_Example_VBNET (Topic)

SOLIDWORKS API Help

Scale Hatch Pattern to Section View Example (VB.NET)

This example shows how to scale a hatch pattern to a section view.

```
'-----  
' Preconditions:  
' 1. Open public_documents\introsw\bolt-assembly.slddrw.  
' 2. Open the Immediate window.  
'-----  
' Postconditions:  
' 1. Scales the hatch pattern to the section view.  
' 2. Examine the Immediate window.  
'-----  
' NOTE: Because this drawing is used elsewhere, do not save changes.  
'-----  
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System  
Imports System.Diagnostics  
  
Partial Class SolidWorksMacro  
  
    Dim swModel As ModelDoc2  
    Dim swDrawing As DrawingDoc  
    Dim swView As View  
    Dim swSectionView As DrSection  
  
    Sub main()  
  
        swModel = swApp.ActiveDoc  
        swDrawing = swModel  
        swDrawing.ActivateView("Drawing View5")  
        swView = swDrawing.ActiveDrawingView  
        swSectionView = swView.GetSection  
        swSectionView.ScaleHatchPattern = True  
        Debug.Print(" Scale hatch pattern to section view? " & swSectionView.ScaleHatchPattern)  
  
    End Sub  
  
    ''' <summary>  
    ''' The SldWorks swApp variable is pre-assigned for you.  
    ''' </summary>  
    Public swApp As SldWorks  
  
End Class
```

Select All Center Marks Example (C#)

In the SOLIDWORKS API, center marks can be features or annotations. Use these enumerators with `ISelectionMgr::GetSelectedObjectType3` to return that specific type of center mark:

- `swSelCENTERMARKS` for center marks that are features
- `swSelCENTERMARKSSYMS` for center marks that are annotations

Traversal over the two types is different for the center marks that are:

- Annotations. Use `IView::GetFirstCenterMark` and `ICenterMark::GetNext`
- Features. Use `IView::GetCenterMarks`.

It is not directly possible to select center marks that are features, even though they appear in a feature traversal. This example shows how to indirectly select center marks that are features.

```
//-----  
// Preconditions:  
// 1. Verify that the drawing document to open exists.  
// 2. Open the Immediate window.  
//  
// Postconditions:  
// 1. Opens the specified drawing document.  
// 2. Prints the path and file name of the drawing document to  
//    the Immediate window.  
// 3. Iterates the sheet and drawing views.  
// 4. Prints the name of the sheet, drawing views,  
//    and center mark annotations to the Immediate window.  
// 5. Iterates the FeatureManager design tree.  
// 6. Prints to the Immediate window that no center mark  
//    features exist in the drawing document.  
//-----  
  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
using System;  
using System.Diagnostics;  
  
namespace CenterMarkCSharp.csproj  
{  
    public partial class SolidWorksMacro  
    {  
        public void Main()  
        {  
            ModelDoc2 swModel = default(ModelDoc2);  
            ModelDocExtension swModExt = default(ModelDocExtension);  
            DrawingDoc swDrawing = default(DrawingDoc);  
            View swView = default(View);  
            CenterMark swCtrMark = default(CenterMark);  
            Annotation swAnn = default(Annotation);  
            Feature swFeat = default(Feature);  
            Feature swSubFeat = default(Feature);  
            Feature swSubSubFeat = default(Feature);  
            SelectionMgr swSelMgr = default(SelectionMgr);  
            SelectData swSelData = default(SelectData);  
            bool status = false;  
            string fileName = null;  
            int errors = 0;  
            int warnings = 0;  
            int i = 0;  
  
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS  
2025\\samples\\tutorial\\advdrawings\\foodprocessor.slddrw";  
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocDRAWING, (int)swOpenDocOption  
s_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);  
            swModExt = (ModelDocExtension)swModel.Extension;  
            swSelMgr = (SelectionMgr)swModel.SelectionManager;  
            swSelData = (SelectData)swSelMgr.CreateSelectData();  
            swDrawing = (DrawingDoc)swModel;  
            swView = (View)swDrawing.GetFirstView();  
            swModel.ClearSelection2(true);  
            Debug.Print("File = " + swModel.GetPathName());  
            Debug.Print("-----");  
  
            i = 0;  
            while ((swView != null))  
            {  
                Debug.Print("  View = " + swView.Name);  
                // Traverse over annotation center marks
```

```

        swCtrMark = (CenterMark)swView.GetFirstCenterMark2();
        while ((swCtrMark != null))
        {
            swAnn = (Annotation)swCtrMark.GetAnnotation();
            Debug.Print(" " + swAnn.GetName());
            // Select directly through annotation
            status = swAnn.Select3(true, swSelData);
            Debug.Assert(status);
            swCtrMark = (CenterMark)swCtrMark.GetNext();
            i = i + 1;
        }
        swView = (View)swView.GetNextView();
    }

    if (i == 0)
    {
        Debug.Print("No center mark annotations.");
    }

    Debug.Print("-----");
    // Traverse over feature center marks
    i = 0;
    swFeat = (Feature)swModel.FirstFeature();
    while ((swFeat != null))
    {
        swSubFeat = (Feature)swFeat.GetFirstSubFeature();
        while ((swSubFeat != null))
        {
            swSubSubFeat = (Feature)swSubFeat.GetFirstSubFeature();
            while ((swSubSubFeat != null))
            {
                if ("CenterMark" == swSubSubFeat.GetTypeName2())
                {
                    Debug.Print(" " + swSubSubFeat.Name);
                    // Cannot directly select feature because feature does not
                    // explicitly appear in FeatureManager design tree
                    // Must indirectly select feature through user interface
                    status = swModExt.SelectByID2(swSubSubFeat.Name, "CENTERMARKS", 0.0, 0.0, 0.0, true, 0,
null, (int)swSelectOption_e.swSelectOptionDefault);
                    Debug.Assert(status);
                }
                swSubSubFeat = (Feature)swSubSubFeat.GetNextSubFeature();
            }
            swSubFeat = (Feature)swSubFeat.GetNextSubFeature();
        }
        swFeat = (Feature)swFeat.GetNextFeature();
    }
    if (i == 0)
    {
        Debug.Print("No center mark features.");
    }

}

}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Select_All_Center_Marks_Example_VB (Topic)

SOLIDWORKS API Help

Select All Center Marks Example (VBA)

In the SOLIDWORKS API, center marks can be features or annotations. Use these enumerators with `ISelectionMgr::GetSelectedObjectType3` to return that specific type of center mark:

- `swSelCENTERMARKS` for center marks that are features
- `swSelCENTERMARKSSYMS` for center marks that are annotations

Traversal over the two types is different for the center marks that are:

- Annotations. Use `IView::GetFirstCenterMark` and `ICenterMark::GetNext`
- Features. Use `IView::GetCenterMarks`.

It is not directly possible to select center marks that are features, even though they appear in a feature traversal. This example shows how to indirectly select center marks that are features.

```
'-----  
' Preconditions:  
' 1. Verify that the drawing document to open exists.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Opens the specified drawing document.  
' 2. Prints the path and file name of the drawing document to  
'    the Immediate window.  
' 3. Iterates the sheet and drawing views.  
' 4. Prints the name of the sheet, drawing views,  
'    and center mark annotations to the Immediate window.  
' 5. Iterates the FeatureManager design tree.  
' 6. Prints to the Immediate window that no center mark  
'    features exist in the drawing document.  
'-----
```

Option Explicit

Sub main()

```
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swModExt As SldWorks.ModelDocExtension  
    Dim swDrawing As SldWorks.DrawingDoc  
    Dim swView As SldWorks.View  
    Dim swCtrMark As SldWorks.CenterMark  
    Dim swAnn As SldWorks.Annotation  
    Dim swFeat As SldWorks.Feature  
    Dim swSubFeat As SldWorks.Feature  
    Dim swSubSubFeat As SldWorks.Feature  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swSelData As SldWorks.SelectData  
    Dim status As Boolean  
    Dim fileName As String  
    Dim errors As Long  
    Dim warnings As Long  
    Dim i As Long  
  
    Set swApp = Application.SldWorks  
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS  
2025\samples\tutorial\advdrawings\foodprocessor.slddrw"  
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING,  
swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)  
    Set swModExt = swModel.Extension  
    Set swSelMgr = swModel.SelectionManager  
    Set swSelData = swSelMgr.CreateSelectData  
    Set swDrawing = swModel  
    Set swView = swDrawing.GetFirstView  
  
    swModel.ClearSelection2 True  
  
    Debug.Print "File = " & swModel.GetPathName  
    Debug.Print "-----"
```

```

i = 0
Do While Not swView Is Nothing
    Debug.Print " View = " & swView.Name
    ' Traverse over annotation center marks
    Set swCtrMark = swView.GetFirstCenterMark2
    Do While Not swCtrMark Is Nothing
        Set swAnn = swCtrMark.GetAnnotation
        Debug.Print " " & swAnn.GetName
        ' Select directly through annotation
        status = swAnn.Select3(True, swSelData): Debug.Assert status
        Set swCtrMark = swCtrMark.GetNext
        i = i + 1
    Loop
    Set swView = swView.GetNextView
Loop

If i = 0 Then
    Debug.Print "No center mark annotations."
End If

Debug.Print "-----"

' Traverse over feature center marks
i = 0
Set swFeat = swModel.FirstFeature
While Not swFeat Is Nothing
    Set swSubFeat = swFeat.GetFirstSubFeature
    While Not swSubFeat Is Nothing
        Set swSubSubFeat = swSubFeat.GetFirstSubFeature
        While Not swSubSubFeat Is Nothing
            If "CenterMark" = swSubSubFeat.GetTypeName2 Then
                Debug.Print " " & swSubSubFeat.Name
                ' Cannot directly select feature because feature does not
                ' explicitly appear in FeatureManager design tree
                ' Must indirectly select feature through user interface
                status = swModExt.SelectByID2(swSubSubFeat.Name, "CENTERMARKS", 0#, 0#, 0#, True, 0, Nothing,
swSelectOptionDefault): Debug.Assert status
                End If
                Set swSubSubFeat = swSubSubFeat.GetNextSubFeature
            Wend
            Set swSubFeat = swSubFeat.GetNextSubFeature
        Wend
        Set swFeat = swFeat.GetNextFeature
    Wend
End While
If i = 0 Then
    Debug.Print "No center mark features."
End If

End Sub

```

Select All Center Marks Example (VB.NET)

In the SOLIDWORKS API, center marks can be features or annotations. Use these enumerators with `ISelectionMgr::GetSelectedObjectType3` to return that specific type of center mark:

- `swSelCENTERMARKS` for center marks that are features
- `swSelCENTERMARKSSYMS` for center marks that are annotations

Traversal over the two types is different for the center marks that are:

- Annotations. Use `IView::GetFirstCenterMark` and `ICenterMark::GetNext`
- Features. Use `IView::GetCenterMarks`.

It is not directly possible to select center marks that are features, even though they appear in a feature traversal. This example shows how to indirectly select center marks that are features.

```
'-----
' Preconditions:
' 1. Verify that the drawing document to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified drawing document.
' 2. Prints the path and file name of the drawing document to
'    the Immediate window.
' 3. Iterates the sheet and drawing views.
' 4. Prints the name of the sheet, drawing views,
'    and center mark annotations to the Immediate window.
' 5. Iterates the FeatureManager design tree.
' 6. Prints to the Immediate window that no center mark
'    features exist in the drawing document.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModExt As ModelDocExtension
        Dim swDrawing As DrawingDoc
        Dim swView As View
        Dim swCtrMark As CenterMark
        Dim swAnn As Annotation
        Dim swFeat As Feature
        Dim swSubFeat As Feature
        Dim swSubSubFeat As Feature
        Dim swSelMgr As SelectionMgr
        Dim swSelData As SelectData
        Dim status As Boolean
        Dim fileName As String
        Dim errors As Integer
        Dim warnings As Integer
        Dim i As Integer

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2025\samples\tutorial\advdrawings\foodprocessor.slddrw"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Sile
nt, "", errors, warnings)
        swModExt = swModel.Extension
        swSelMgr = swModel.SelectionManager
        swSelData = swSelMgr.CreateSelectData
        swDrawing = swModel
        swView = swDrawing.GetFirstView
        swModel.ClearSelection2(True)
        Debug.Print("File = " & swModel.GetPathName)
        Debug.Print("-----")

        i = 0
        Do While Not swView Is Nothing
            Debug.Print(" View = " & swView.Name)
            ' Traverse over annotation center marks
            swCtrMark = swView.GetFirstCenterMark2
            Do While Not swCtrMark Is Nothing
                swAnn = swCtrMark.GetAnnotation
                Debug.Print(" " & swAnn.GetName)
                ' Select directly through annotation
```

```

        status = swAnn.Select3(True, swSelData) : Debug.Assert(status)
        swCtrMark = swCtrMark.GetNext
        i = i + 1
    Loop
    swView = swView.GetNextView
Loop

If i = 0 Then
    Debug.Print("No center mark annotations.")
End If

Debug.Print("-----")
' Traverse over feature center marks
i = 0
swFeat = swModel.FirstFeature
While Not swFeat Is Nothing
    swSubFeat = swFeat.GetFirstSubFeature
    While Not swSubFeat Is Nothing
        swSubSubFeat = swSubFeat.GetFirstSubFeature
        While Not swSubSubFeat Is Nothing
            If "CenterMark" = swSubSubFeat.GetTypeName2 Then
                Debug.Print(" " & swSubSubFeat.Name)
                ' Cannot directly select feature because does not
                ' explicitly appear in FeatureManager design tree
                ' Must indirectly select feature through user interface
                status = swModExt.SelectByID2(swSubSubFeat.Name, "CENTERMARKS", 0.0#, 0.0#, 0.0#, True, 0,
Nothing, swSelectOption_e.swSelectOptionDefault) : Debug.Assert(status)
            End If
            swSubSubFeat = swSubSubFeat.GetNextSubFeature
        End While
        swSubFeat = swSubFeat.GetNextSubFeature
    End While
    swFeat = swFeat.GetNextFeature
End While
If i = 0 Then
    Debug.Print("No center mark features.")
End If

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Select_All_Faces_on_Part_Example_VB (Topic)

SOLIDWORKS API Help

Select All Faces on Part Example (VBA)

This example shows how to get all of the faces in a part.

```
'-----  
' Preconditions: Open a part document containing one part.  
'  
' Postconditions:  
' 1. Selects all faces on the part.  
' 2. Examine the graphics area.  
'-----
```

Option Explicit

Sub main()

```
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swSelData As SldWorks.SelectData  
    Dim swPart As SldWorks.PartDoc  
    Dim swBody As SldWorks.Body2  
    Dim swFace As SldWorks.Face  
    Dim swEnt As SldWorks.Entity  
    Dim bRet As Boolean  
    Dim vBodies As Variant
```

```
    Set swApp = CreateObject("SldWorks.Application")  
    Set swModel = swApp.ActiveDoc  
    Set swPart = swModel  
    Set swSelMgr = swModel.SelectionManager  
    Set swSelData = swSelMgr.CreateSelectData  
    vBodies = swPart.GetBodies2(swAllBodies, True)  
    Set swBody = vBodies(0)  
    Set swFace = swBody.GetFirstFace
```

```
    swModel.ClearSelection2 True
```

```
    Do While Not swFace Is Nothing  
        Set swEnt = swFace  
        ' Select using IEntity  
        bRet = swEnt.Select4(True, swSelData)  
        Set swFace = swFace.GetNextFace  
    Loop
```

End Sub

Select All Features Example (VBA)

This example shows how to select all of the features in a part.

```
'-----  
' Preconditions:  
' 1. Open a part.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Selects all selectable features.  
' 2. Examine the FeatureManager design tree, graphics  
'    area, and Immediate window.  
'  
' NOTE: Some features are not selectable.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp                As SldWorks.SldWorks  
    Dim swModel              As SldWorks.ModelDoc2  
    Dim swFeatMgr            As SldWorks.FeatureManager  
    Dim swFeat              As SldWorks.Feature  
    Dim swSubFeat           As SldWorks.Feature  
    Dim swSubSubFeat        As SldWorks.Feature  
    Dim swSubSubSubFeat     As SldWorks.Feature  
    Dim i                   As Long  
    Dim bRet                As Boolean  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swFeatMgr = swModel.FeatureManager  
  
    ' Disable FeatureManager design tree to increase performance  
    swFeatMgr.EnableFeatureTree = False  
  
    swModel.ClearSelection2 True  
  
    Debug.Print "File = " & swModel.GetPathName  
  
    Set swFeat = swModel.FirstFeature  
    Do While Not swFeat Is Nothing  
        bRet = swFeat.Select2(True, 0)  
        Debug.Print " " & swFeat.Name & " [" & swFeat.GetTypeName & "]"  
        Set swSubFeat = swFeat.GetFirstSubFeature  
        Do While Not swSubFeat Is Nothing  
            bRet = swSubFeat.Select2(True, 0)  
            Debug.Print " " & swSubFeat.Name & " [" & swSubFeat.GetTypeName & "]"  
            Set swSubSubFeat = swSubFeat.GetFirstSubFeature  
            Do While Not swSubSubFeat Is Nothing  
                bRet = swSubSubFeat.Select2(True, 0)  
                Debug.Print " " & swSubSubFeat.Name & " [" & swSubSubFeat.GetTypeName & "]"  
                Set swSubSubSubFeat = swSubSubFeat.GetFirstSubFeature  
                Do While Not swSubSubSubFeat Is Nothing  
                    bRet = swSubSubSubFeat.Select2(True, 0)  
                    Debug.Print " " & swSubSubSubFeat.Name & " [" & swSubSubSubFeat.GetTypeName & "]"  
                    Set swSubSubSubFeat = swSubSubSubFeat.GetNextSubFeature  
                Loop  
                Set swSubSubFeat = swSubSubFeat.GetNextSubFeature  
            Loop  
            Set swSubFeat = swSubFeat.GetNextSubFeature  
        Loop  
        Set swFeat = swFeat.GetNextFeature  
    Loop  
  
    'bRet = swModel.DeleteSelection(False)  
  
    ' Re-enable FeatureManager design tree to show new selections  
    swFeatMgr.EnableFeatureTree = True
```

End Sub

Select_All_Fillets_Example_VB (Topic)

SOLIDWORKS API Help

Select All Fillets Example (VBA)

Selecting all of the fillets on an active part is done by iterating through all of the features with `IModelDoc2::FirstFeature` and `IFeature::GetNextFeature`. For each feature, check the type with `IFeature::GetTypeName` and compare it to the "Fillet" and "VarFillet" constants to see if the current feature is a fillet. If the feature is a fillet, then place the fillet in an `IEntity` object and select the entity.

```
'-----  
'  
' Preconditions:  
' (1) Part document is open.  
' (2) Fillets exist on the part.  
'  
' Postconditions: All of the fillets are selected.  
'  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim Model As SldWorks.ModelDoc2  
Dim CurFeature As SldWorks.feature  
Dim CurEntity As SldWorks.entity  
Dim SelData As SldWorks.SelectData
```

```
Sub main()  
    ' Get the SOLIDWORKS Design object and the active model  
    Set swApp = GetObject(, "SldWorks.Application")  
    Set Model = swApp.ActiveDoc  
  
    ' Get the first feature on the part  
    Set CurFeature = Model.FirstFeature  
    ' For all of the features on the part  
    Do While Not CurFeature Is Nothing  
        ' If the current feature is one of the fillet types,  
        ' then select the current fillet  
        If CurFeature.GetTypeName = "Fillet" Or CurFeature.GetTypeName = "VarFillet" Then  
            Set CurEntity = CurFeature  
            CurEntity.Select4 True, SelData  
        End If  
        'Go to the next feature  
        Set CurFeature = CurFeature.GetNextFeature  
    Loop  
End Sub
```

Select All in Part, Assembly, or Drawing (C#)

This example shows how to select everything in the graphics area of a part or assembly document or in the sheet of a drawing document, as if you box-selected everything in the graphics area or the sheet.

```
//-----
// Preconditions:
// 1. Verify that the part, assembly, and drawing documents opened by the macro
//    exist.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Examine:
//    * Sheet to verify that all of the entities in the drawing
//      are selected.
//    * Immediate window to see how many entities are selected.
// 2. Click Window > bolt-assembly.sldasm to switch to the assembly
//    document.
// 3. Examine:
//    * Graphics area to verify that the all of the components
//      in the assembly are selected.
//    * Immediate window to see how many components are selected.
// 4. Click Window > bolt.sldprt to switch to the part document.
// 5. Examine:
//    * Graphics area to verify that the all of the edges
//      in the part are selected.
//    * Immediate window to see how many edges are selected.
//
// NOTE: Because these documents are used elsewhere, do not save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;

namespace SelectAllModelDocExtensionCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel;
            ModelDocExtension swModelDocExt;
            SelectionMgr swSelMgr;
            string partFile = null;
            string assemblyFile = null;
            string drawingFile = null;
            int errors = 0;
            int warnings = 0;

            // Open a part document and select all edges in the part
            partFile = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2017\\introsw\\bolt.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(partFile, (int)swDocumentTypes_e.swDocPART,
(int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            //Select all edges in part
            SelectAllinDocument(swModel, swModelDocExt, swSelMgr);

            // Open an assembly document and select all components in the assembly
            assemblyFile = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2017\\introsw\\bolt-assembly.sldasm";
            swModel = (ModelDoc2)swApp.OpenDoc6(assemblyFile,
```

```

(int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref
errors, ref warnings);
    swModelDocExt = (ModelDocExtension)swModel.Extension;
    swSelMgr = (SelectionMgr)swModel.SelectionManager;
    //Select all components in assembly
    SelectAllinDocument(swModel, swModelDocExt, swSelMgr);

    // Open a drawing document and select all entities in the drawing
    drawingFile = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2017\\intros\\bolt-assembly.slddrw";
    swModel = (ModelDoc2)swApp.OpenDoc6(drawingFile, (int)swDocumentTypes_e.swDocDRAWING,
(int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
    swModelDocExt = (ModelDocExtension)swModel.Extension;
    swSelMgr = (SelectionMgr)swModel.SelectionManager;
    //Select all entities in drawing
    SelectAllinDocument(swModel, swModelDocExt, swSelMgr);

}

public void SelectAllinDocument(ModelDoc2 swModel, ModelDocExtension swModelDocExt,
SelectionMgr swSelMgr)
{
    int selCount;

    // Select all edges in a part, all components in an assembly,
    // or all entities in a drawing
    swModelDocExt.SelectAll();

    // Get and print the number of selections
    selCount = 0;
    selCount = swSelMgr.GetSelectedObjectCount2(-1);

    switch (swModel.GetType())
    {
        case (int)swDocumentTypes_e.swDocPART:
            Debug.Print("Number of edges selected in part          = " + selCount);
            break;
        case (int)swDocumentTypes_e.swDocASSEMBLY:
            Debug.Print("Number of components selected in assembly = " + selCount);
            break;
        case (int)swDocumentTypes_e.swDocDRAWING:
            Debug.Print("Number of entities selected in drawing    = " + selCount);
            break;
        default:
            Debug.Print("Unknown type of document.");
            break;
    }

}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Select All in Part, Assembly, or Drawing (VBA)

This example shows how to select everything in the graphics area of a part or assembly document or in the sheet of a drawing document, as if you box-selected everything in the graphics area or the sheet.

```
'-----  
' Preconditions:  
' 1. Verify that the part, assembly, and drawing documents opened by the macro  
'    exist.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Examine:  
'    * Sheet to verify that all of the entities in the drawing  
'      are selected.  
'    * Immediate window to see how many entities are selected.  
' 2. Click Window > bolt-assembly.sldasm to switch to the assembly  
'    document.  
' 3. Examine:  
'    * Graphics area to verify that the all of the components  
'      in the assembly are selected.  
'    * Immediate window to see how many components are selected.  
' 4. Click Window > bolt.sldprt to switch to the part document.  
' 5. Examine:  
'    * Graphics area to verify that the all of the edges  
'      in the part are selected.  
'    * Immediate window to see how many edges are selected.  
'  
' NOTE: Because these documents are used elsewhere, do not save changes.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim partFile As String  
Dim assemblyFile As String  
Dim drawingFile As String  
Dim errors As Long  
Dim warnings As Long  
Dim selCount As Integer  
  
Sub main()  
    Set swApp = Application.SldWorks  
  
    ' Open a part document and select all edges in the part  
    partFile = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2017\introsw\bolt.sldprt"  
    Set swModel = swApp.OpenDoc6(partFile, swDocPART, swOpenDocOptions_Silent, "", errors, warnings)  
    Set swModelDocExt = swModel.Extension  
    Set swSelMgr = swModel.SelectionManager  
    'Select all edges in part  
    SelectAllinDocument  
  
    ' Open an assembly document and select all components in the assembly  
    assemblyFile = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2017\introsw\bolt-assembly.sldasm"  
    Set swModel = swApp.OpenDoc6(assemblyFile, swDocASSEMBLY, swOpenDocOptions_Silent, "", errors, warnings)  
    Set swModelDocExt = swModel.Extension  
    Set swSelMgr = swModel.SelectionManager  
    'Select all components in assembly  
    SelectAllinDocument  
  
    ' Open a drawing document and select all entities in the drawing  
    drawingFile = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2017\introsw\bolt-assembly.slddrw"  
    Set swModel = swApp.OpenDoc6(drawingFile, swDocDRAWING, swOpenDocOptions_Silent, "", errors, warnings)  
    Set swModelDocExt = swModel.Extension  
    Set swSelMgr = swModel.SelectionManager
```

```
'Select all entities in drawing  
SelectAllinDocument
```

```
End Sub
```

```
Sub SelectAllinDocument()  
' Select all edges in a part, all components in an assembly,  
' or all entities in a drawing  
swModelDocExt.SelectAll  
  
' Get and print the number of selections  
selCount = 0  
selCount = swSelMgr.GetSelectedObjectCount2(-1)  
  
Select Case swModel.GetType  
Case swDocPART  
    Debug.Print "Number of edges selected in part          = " & selCount  
Case swDocASSEMBLY  
    Debug.Print "Number of components selected in assembly = " & selCount  
Case swDocDRAWING  
    Debug.Print "Number of entities selected in drawing    = " & selCount  
Case Else  
    Debug.Print "Unknown type of document."  
End Select
```

```
End Sub
```

Select All in Part, Assembly, or Drawing (VB.NET)

This example shows how to select everything in the graphics area of a part or assembly document or in the sheet of a drawing document, as if you box-selected everything in the graphics area or the sheet.

```
'-----  
' Preconditions:  
' 1. Part, assembly, and drawing documents opened by the macro  
'    exist.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Examine:  
'    * Sheet to verify that all of the entities in the drawing  
'      are selected.  
'    * Immediate window to see how many entities are selected.  
' 2. Click Window > bolt-assembly.sldasm to switch to the assembly  
'    document.  
' 3. Examine:  
'    * Graphics area to verify that the all of the components  
'      in the assembly are selected.  
'    * Immediate window to see how many components are selected.  
' 4. Click Window > bolt.sldprt to switch to the part document.  
' 5. Examine:  
'    * Graphics area to verify that the all of the edges  
'      in the part are selected.  
'    * Immediate window to see how many edges are selected.  
'  
' NOTE: Because these documents are used elsewhere, do not save changes.  
'-----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System
```

```
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Sub Main()
```

```
        Dim swModel As ModelDoc2
```

```
        Dim swModelDocExt As ModelDocExtension
```

```
        Dim swSelMgr As SelectionMgr
```

```
        Dim partFile As String
```

```
        Dim assemblyFile As String
```

```
        Dim drawingFile As String
```

```
        Dim errors As Integer
```

```
        Dim warnings As Integer
```

```
        ' Open a part document and select all edges in the part
```

```
        partFile = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2017\introsw\bolt.sldprt"
```

```
        swModel = swApp.OpenDoc6(partFile, swDocumentTypes_e.swDocPART,  
swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
```

```
        swModelDocExt = swModel.Extension
```

```
        swSelMgr = swModel.SelectionManager
```

```
        'Select all edges in part
```

```
        SelectAllinDocument(swModel, swModelDocExt, swSelMgr)
```

```
        ' Open an assembly document and select all components in the assembly
```

```
        assemblyFile = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2017\introsw\bolt-  
assembly.sldasm"
```

```
        swModel = swApp.OpenDoc6(assemblyFile, swDocumentTypes_e.swDocASSEMBLY,  
swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
```

```
        swModelDocExt = swModel.Extension
```

```
        swSelMgr = swModel.SelectionManager
```

```
        'Select all components in assembly
```



```

        SelectAllinDocument(swModel, swModelDocExt, swSelMgr)

        ' Open a drawing document and select all entities in the drawing
        drawingFile = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2017\introsw\bolt-
assembly.slddrw"
        swModel = swApp.OpenDoc6(drawingFile, swDocumentTypes_e.swDocDRAWING,
swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModelDocExt = swModel.Extension
        swSelMgr = swModel.SelectionManager
        'Select all entities in drawing
        SelectAllinDocument(swModel, swModelDocExt, swSelMgr)

End Sub

Sub SelectAllinDocument(ByVal swModel As ModelDoc2, ByVal swModelDocExt As ModelDocExtension,
ByVal swSelMgr As SelectionMgr)
    Dim selCount As Integer

    ' Select all edges in a part, all components in an assembly,
    ' or all entities in a drawing
    swModelDocExt.SelectAll()

    ' Get and print the number of selections
    selCount = 0
    selCount = swSelMgr.GetSelectedObjectCount2(-1)

    Select Case swModel.GetType
        Case swDocumentTypes_e.swDocPART
            Debug.Print("Number of edges selected in part          = " & selCount)
        Case swDocumentTypes_e.swDocASSEMBLY
            Debug.Print("Number of components selected in assembly = " & selCount)
        Case swDocumentTypes_e.swDocDRAWING
            Debug.Print("Number of entities selected in drawing    = " & selCount)
        Case Else
            Debug.Print("Unknown type of document.")
    End Select

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Select All Sketch Segments Example (C++ COM)

This example shows how to use the IEnumSketchSegments interface to select all sketch segments in the active sketch. Nearly identical code could be used to also select all the sketch points in the active sketch by replacing IEnumSketchSegments with IEnumSketchPoints.

```
LPMODELDOC pModelDoc = NULL;
hres = UserApp->getSWApp()->get_IActiveDoc(&pModelDoc);
if (pModelDoc == NULL) return;

LPSKETCH pSketch = NULL;
hres = pModelDoc->IGetActiveSketch2(&pSketch);

// No Active Sketch
if (pSketch == NULL) return;

LPENUMSKETCHSEGMENTS pSketchSegEnum = NULL;
hres = pSketch->IEnumSketchSegments(&pSketchSegEnum);

// No Sketch segments exist
if (pSketchSegEnum == NULL) return;

int segNum = 0;
LPSKETCHSEGMENT pSketchSegment = NULL;
long skReturned = -1
hres = pSketchSegEnum->Next(1, &pSketchSegment, &skReturned);
while (S_OK == hres)
{
    VARIANT_BOOL ok = FALSE;

    // Select the SketchSegment object
    hres = pSketchSegment->Select(TRUE, &ok);

    if (!ok) AfxMessageBox(_T("Error Selecting Sketch Segment! Continuing..."));

    pSketchSegment->Release();
    pSketchSegment = NULL;
    hres = pSketchSegEnum->Next(1, &pSketchSegment, &skReturned);
}

if (pSketchSegEnum != NULL) pSketchSegEnum->Release();
```

Select Assembly Components by Size Example (C#)

This example shows how to select components by percent of assembly size.

```
//-----  
// Preconditions: Open an assembly.  
//  
// Postconditions: Selects components that are 30% of the size of the  
// assembly or smaller.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
using System;  
namespace SelectComponentsBySize_CSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
        ModelDoc2 Part;  
        AssemblyDoc assem;  
        bool result;  
  
        public void Main()  
        {  
            Part = (ModelDoc2)swApp.ActiveDoc;  
            assem = (AssemblyDoc)Part;  
            result = assem.SelectComponentsBySize(30.0);  
        }  
  
        public SldWorks swApp;  
    }  
}
```

Select_Assembly_Components_by_Size_Example_VB (Topic)

SOLIDWORKS API Help

Select Assembly Components by Size Example (VBA)

This example shows how to select components by percent of assembly size.

```
'-----  
' Preconditions: Open an assembly.  
'  
' Postconditions: Selects components that are 30% of the size of the  
' assembly or smaller.  
'-----  
  
Dim swApp As SldWorks.SldWorks  
Dim Part As SldWorks.ModelDoc2  
Dim assem As SldWorks.AssemblyDoc  
Dim bool As Boolean  
  
Option Explicit  
  
Sub main()  
    Set swApp = Application.SldWorks  
    Set Part = swApp.ActiveDoc  
    Set assem = Part  
    bool = assem.SelectComponentsBySize(30#)  
  
End Sub
```

Select_Assembly_Components_by_Size_Example_VBNET (Topic)

SOLIDWORKS API Help

Select Assembly Components by Size Example (VB.NET)

This example shows how to select components by percent of assembly size.

```
'-----  
' Preconditions: Open an assembly.  
'  
' Postconditions: Selects components that are 30% of the size of the  
' assembly or smaller.  
'-----  
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System  
  
Partial Class SolidWorksMacro  
  
    Dim Part As ModelDoc2  
    Dim assem As AssemblyDoc  
    Dim bool As Boolean  
  
    Sub main()  
  
        Part = swApp.ActiveDoc  
        assem = Part  
        bool = assem.SelectComponentsBySize(30.0#)  
  
    End Sub  
  
    Public swApp As SldWorks  
  
End Class
```